LAPORAN TUGAS KELOMPOK - WALAWE MID

Pengembangan Aplikasi yang Berjalan pada Container

Aplikasi: **Mangatsu**

Disusun oleh:

Carlos Leonardo J. Dongoran - 231111859

Yohanes Cornelius Wilifredi - 231111676

Radhit Maulana - 231110680

DAFTAR ISI

1. Informasi Aplikasi dan Anggota Kelompok 3

Latar belakang 3

Informasi detail aplikasi 3

Status pengembang 3

2. Deskripsi Singkat dan Tata Cara Penggunaan 4

Tata cara penggunaan 4

3. Informasi Detail Aplikasi 6

Arsitektur container 6

Detail port 7

Teknologi yang digunakan 7

Struktur project 8

Penerapan database 8

Penerapan CRUD pada bookmark 8

Sumber kode, repository, Docker Hub, dan referensi 9

Detail image di Docker Hub 10

4. File Pendukung dan Aplikasi Tambahan 10

Tampilan container 11

5. Link Video Dokumentasi 11

6. Originalitas 11

Update Perubahan 12

1. Ringkasan perubahan 12

2. Latar belakang perubahan API 12

3. Perubahan integrasi dari Tenrai API ke MangaDex API 13

4. Build dan menjalankan dengan Docker Compose 15

5. Fitur baru 19

5.1 Reader chapter 19

5.2 Progres dan riwayat baca 20

5.3 Ranking Trending 21

5.4 Ulasan dan rating 21

5.5 Eksplorasi genre 21

5.6 Favorit 22

6. Database terbaru 22

7. Endpoint API setelah update 25

a. Katalog dan pencarian 25

b. Genre 25

c. Chapter dan reader 26

d. Ulasan 26

e. Progres baca dan Trending....26
f. Akun....28
g. Favorit....28
h. Bookmark....28
Implementasi CI/CD....29
1. Tujuan dan alur....29
2. Flowchart....29
3. GitHub Secrets....29
4. Isi file workflow....29
5. Bukti eksekusi....30
6. Cakupan dan batasan CI/CD....32
7. Keterkaitan Before/After dengan bukti pipeline....33

1. Informasi Aplikasi dan Anggota Kelompok

Nama aplikasi	Mangatsu (katalog dan platform baca manga berbahasa Indonesia)
Anggota 1	Carlos Leonardo J. Dongoran - 231111859
Anggota 2	Yohanes Cornelius Wilifredi - 231111676
Anggota 3	Radhit Maulana - 231110680

Latar belakang

Banyak situs manga hanya menyediakan daftar bacaan tanpa cara yang praktis untuk mencatat manga yang sedang, akan, atau sudah selesai dibaca. Mangatsu dibuat untuk mengisi kebutuhan itu: setiap pengguna memiliki akun dan rak buku pribadi yang menyimpan manga beserta status bacanya, dengan tampilan yang nyaman untuk membaca dalam waktu lama. Dari sisi teknis, aplikasi terdiri dari beberapa komponen (frontend, backend, database, dan alat bantu database) yang harus berjalan bersamaan. Karena itu tugas ini memakai Docker dan Docker Compose agar seluruh komponen dapat dibangun dan dijalankan dengan satu perintah, dan menghasilkan lingkungan yang sama di komputer mana pun. Di sisi

Informasi detail aplikasi

Jenis aplikasi	Aplikasi web (Flutter Web) dengan backend REST API
Pengguna sasaran	Pembaca manga berbahasa Indonesia
Fitur utama	Registrasi dan login, pencarian manga (judul, author, genre), halaman detail manga, rekomendasi manga serupa, bookmark dengan status baca (Reading, Plan, Completed), tiga mode tampilan (terang, gelap, sepia), dan hapus akun
Sumber data manga	Tenrai API (api.tenrai.org), disimpan sementara (cache) di server selama 24 jam
Penyimpanan data	MySQL 8.0 (tabel users dan bookmarks)
Cara menjalankan	`docker compose up --build -d`, lalu buka http://localhost:8080

Status pengembang

Aplikasi ini dikembangkan secara mandiri oleh kelompok (tiga anggota di atas). Komponen dari pihak lain hanya berupa data, image, dan library open source yang dipakai sebagai bahan dasar, dengan rincian berikut.

Komponen	Pembuat	Keterangan
Kode frontend (Flutter/Dart)	Kelompok (mandiri)	Folder project_manga/lib
Kode backend (Flask dari Python)	Kelompok (mandiri)	Folder backend_manga/manga_api dan app.py
Skema database	Kelompok (mandiri)	File perpustakaan.sql
Dockerfile, docker-compose.yml, nginx.conf	Kelompok (mandiri)	Konfigurasi container disusun sendiri
Data katalog manga	Pihak lain	Tenrai API (api.tenrai.org), berbasis data MyAnimeList
Image mysql, nginx, python,	Pihak lain	Image resmi dari Docker Hub (link pada bagian 3)

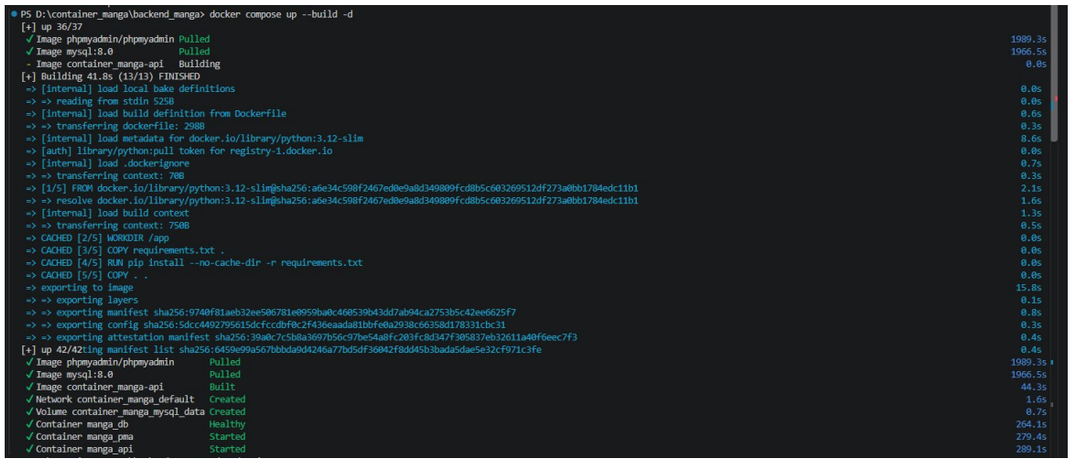
Komponen	Pembuat	Keterangan
phpmyadmin		
Framework dan library	Pihak lain (open source)	Flutter, Flask, Flask-CORS, PyMySQL, Werkzeug, Gunicorn

2. Deskripsi Singkat dan Tata Cara Penggunaan

Mangatsu adalah aplikasi web untuk menjelajahi buku manga dan menyimpan manga ke rak buku pribadi. Pengguna dapat mendaftar, login, mencari manga berdasarkan judul, author, atau genre, melihat detail manga, lalu menyimpannya sebagai bookmark dengan status baca (Reading, Plan, Completed). Fitur CRUD diterapkan pada akun pengguna dan bookmark, dan seluruh data disimpan di database MySQL.

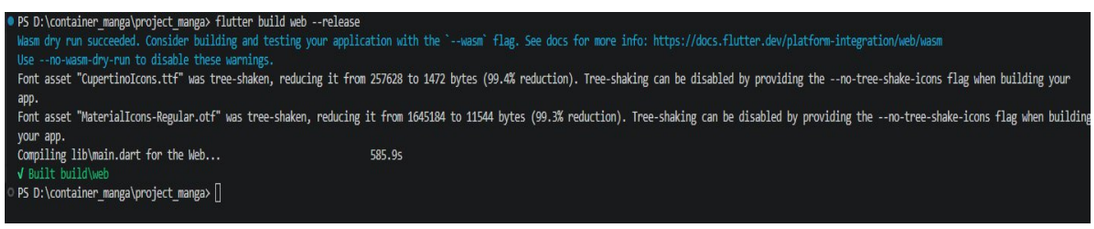
Tata cara penggunaan

1. Jalankan seluruh container dengan perintah `docker compose up --build -d` dari folder project.
   a) Proses pembuatan container pada backend sebagai berikut:

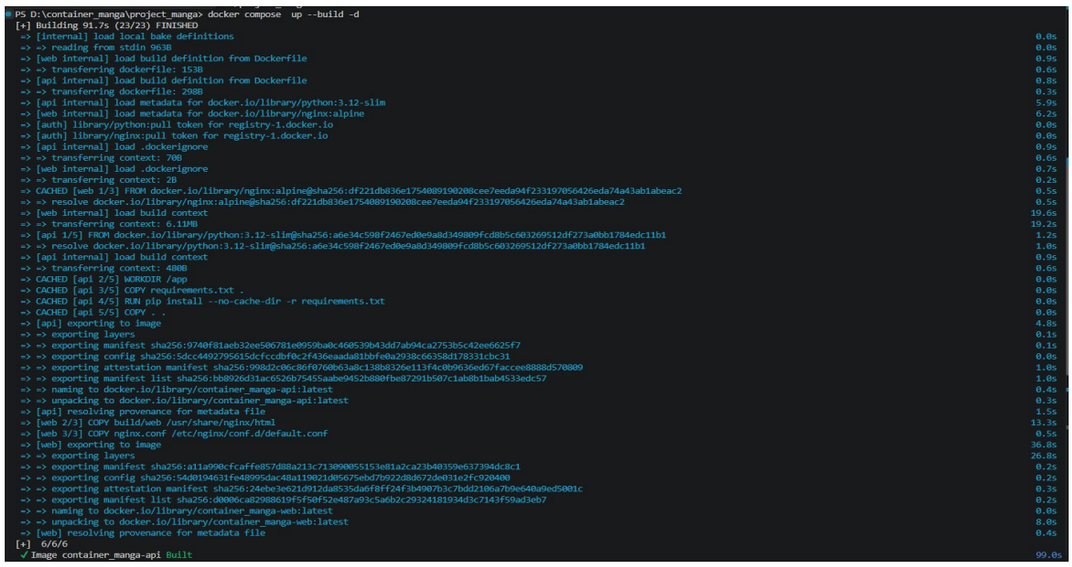


Gambar 1 container backend

   b) Proses pembuatan build web dan build container pada frontend (tampilan website manga) sebagai berikut:

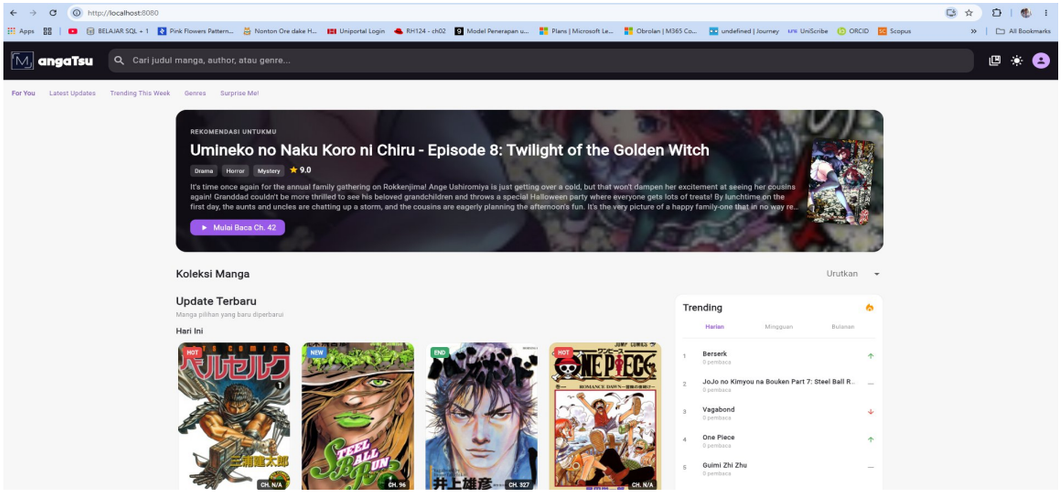


Gambar 2 Proses Build_web_release



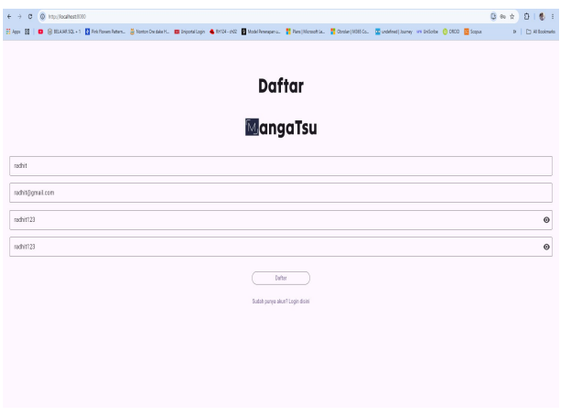
Gambar 3 Container frontend

2. Buka http://localhost:8080 untuk aplikasi (Flutter Web).

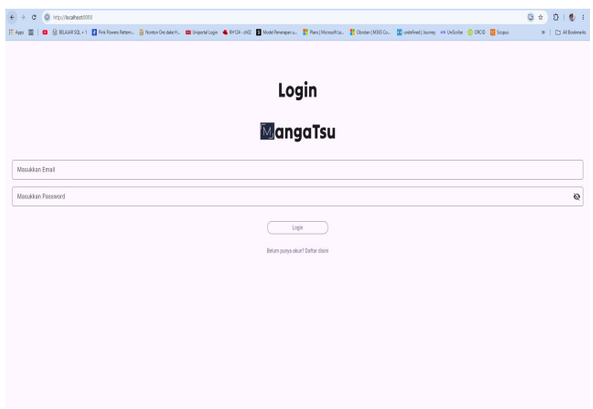


Gambar 4 tampilan web

3. Daftar akun baru (Create), lalu login.

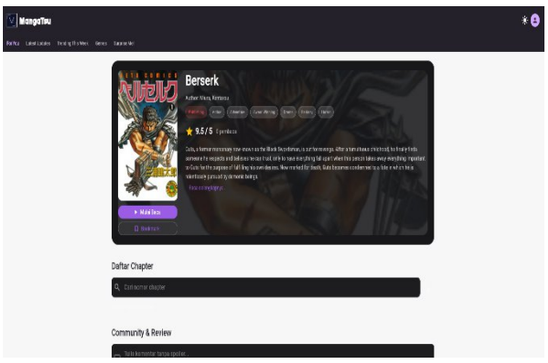


Gambar 5 Daftar akun

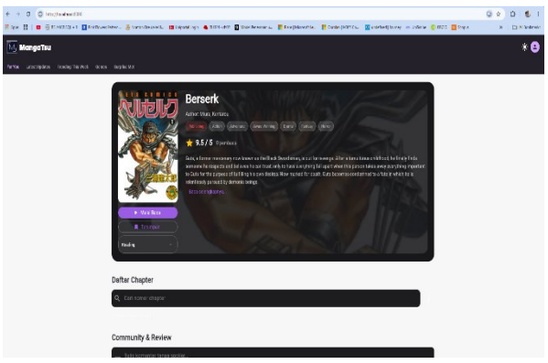


Gambar 6 Login akun

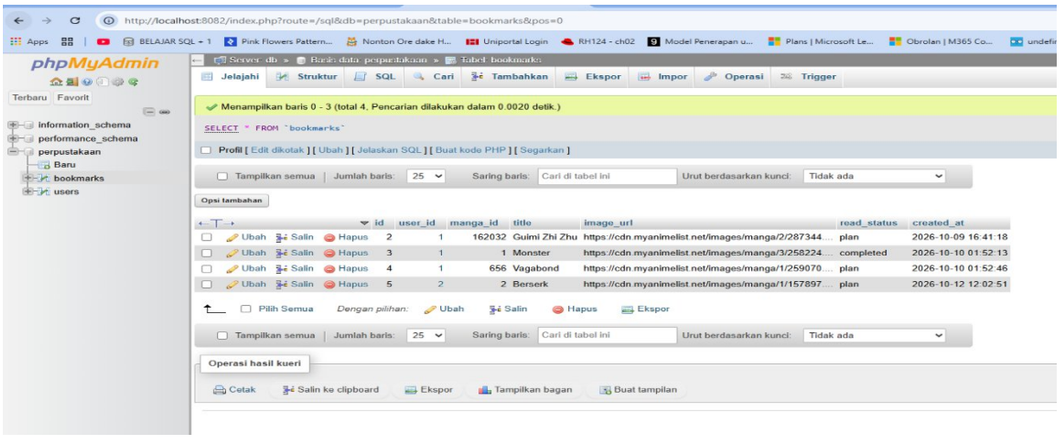
4. Cari atau pilih manga, buka halaman detail, klik *Bookmark* (Create).



Gambar 7 Tampilan awal halaman detail

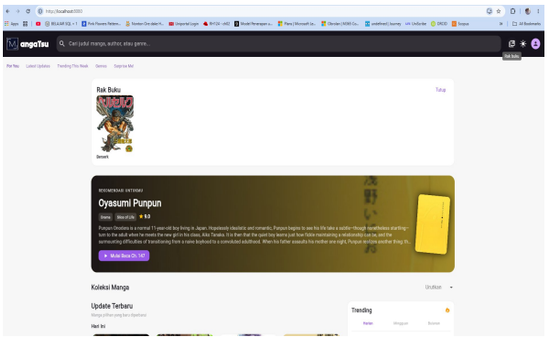


Gambar 8 Perubahan setelah bookmark diklik

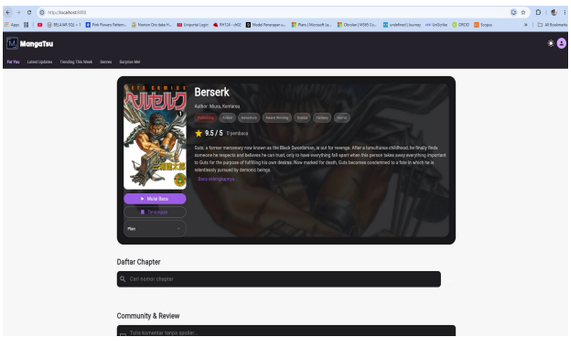


Gambar 9 hasil perubahan di database

5. Lihat koleksi di rak buku (Read) dan ubah status baca dari halaman detail (Update).

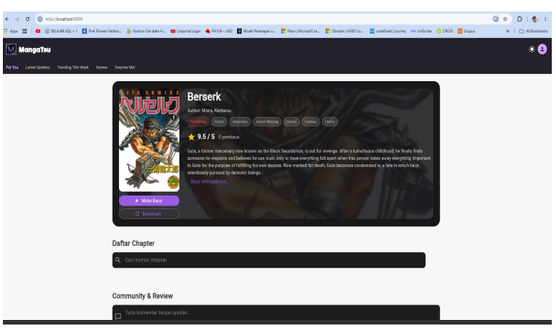


Gambar 10 Isi tampilan di rak buku

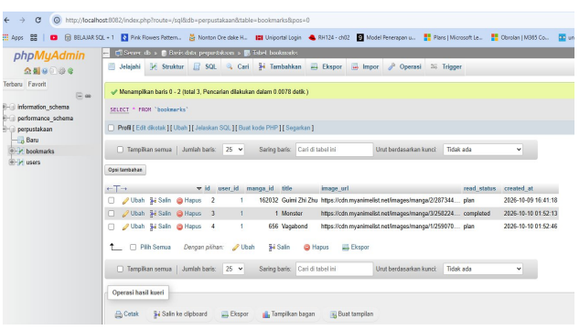


Gambar 11 Isi detail setelah status diedit dari halaman detail

6. Klik bookmark kembali untuk menghapus dari koleksi (Delete). Akun juga dapat dihapus dari halaman Profil dengan konfirmasi password.



Gambar 12 Tampilan sesudah diklik tersimpan



Gambar 13 Perubahan di database

7. Periksa perubahan data di phpMyAdmin pada http://localhost:8082.

3. Informasi Detail Aplikasi

Arsitektur container

Aplikasi terdiri dari empat container yang dijalankan bersama melalui Docker Compose dan saling terhubung lewat jaringan internal Docker.

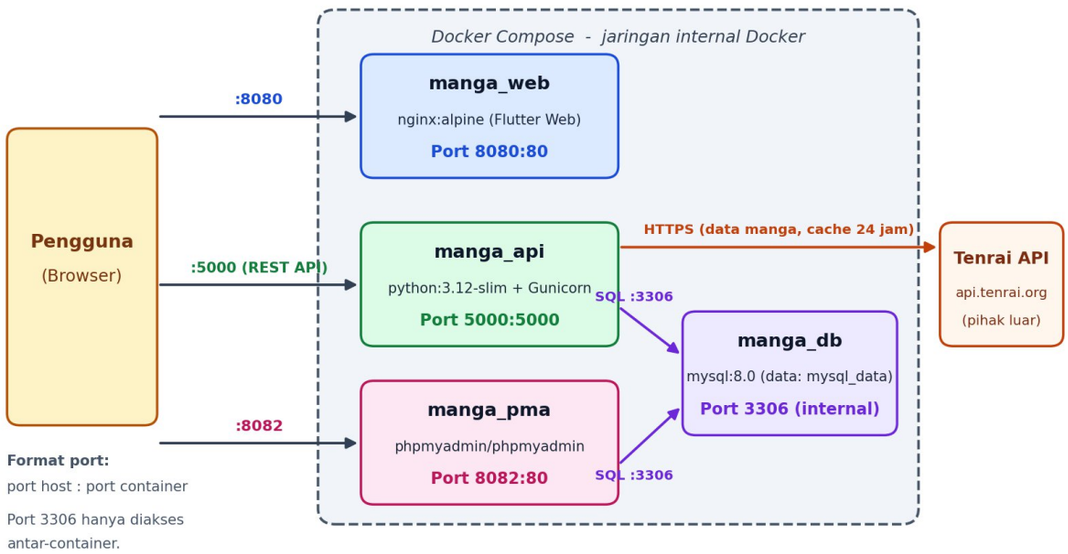


Diagram 1 Arsitektur dan port container

```
PS D:\container_manga\backend_manga> docker compose ps
NAME        IMAGE                                                                  COMMAND                  SERVICE     CREATED       STATUS                   PORTS
manga_api   sha256:6459e99a567bbbda9d4246a77bd5df36042f8dd45b3bada5dae5e32cf971c3fe "gunicorn -w 2 -b 0.…"   api         4 hours ago   Up 3 hours               0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp
manga_db    mysql:8.0                                                              "docker-entrypoint.s…"   db          4 hours ago   Up 4 hours (healthy)     3306/tcp, 33060/tcp
manga_pma   phpmyadmin/phpmyadmin                                                  "/docker-entrypoint.…"   phpmyadmin  4 hours ago   Up 3 hours               0.0.0.0:8082->80/tcp, [::]:8082->80/tcp
manga_web   container_manga-web                                                    "/docker-entrypoint.…"   web         3 hours ago   Up 3 hours               0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
PS D:\container_manga\backend_manga>
```

Container	Image / sumber	Port (host:container)	Fungsi
manga_db	mysql:8.0	3306 (internal)	Database MySQL. Tabel *users* dan *bookmarks* dibuat otomatis dari file perpustakaan.sql. Data disimpan pada volume mysql_data.
manga_api	Build dari backend_manga/Dockerfile (python:3.12-slim + Gunicorn)	5000:5000	Backend Flask: registrasi, login, hapus akun, bookmark (CRUD), dan data manga.
manga_web	Build dari Dockerfile frontend (nginx:alpine)	8080:80	Menyajikan hasil build Flutter Web.
manga_pma	phpmyadmin/phpmyadmin	8082:80	Antarmuka untuk memeriksa isi database.

Detail port

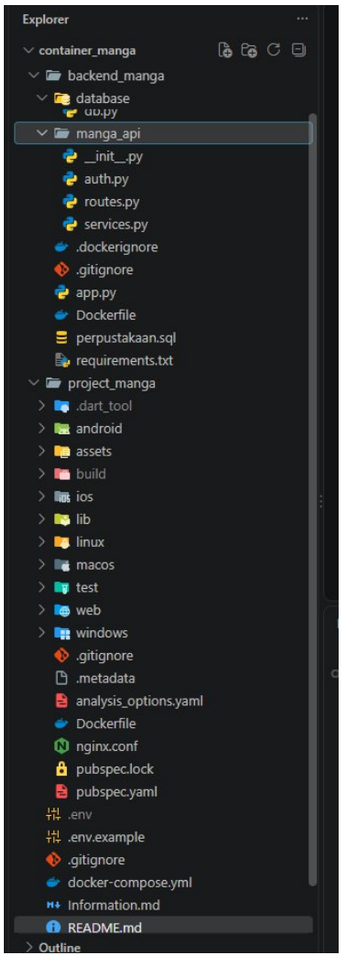
manga_web memetakan port 8080 di komputer ke port 80 di dalam container (Nginx), sehingga aplikasi dibuka melalui http://localhost:8080. manga_api memetakan port 5000 ke port 5000 (Gunicorn) dan menjadi alamat REST API di http://localhost:5000. manga_pma memetakan port 8082 ke port 80 (phpMyAdmin), sehingga phpMyAdmin dibuka lewat alamat http://localhost:8082 di

browser. manga_db memakai port 3306 (MySQL) hanya di dalam jaringan internal Docker, tidak dibuka ke komputer, dan diakses oleh manga_api serta manga_pma.

Teknologi yang digunakan

Frontend: Flutter (Dart) dibangun menjadi web. Backend: Python Flask dengan Flask-CORS, PyMySQL, dan Werkzeug (hash password). Database: MySQL 8.0. Server web: Nginx. Server aplikasi: Gunicorn. Container: Docker dan Docker Compose.

Struktur project



backend_manga/ berisi kode backend Flask: database/db.py (koneksi database), manga_api/ (auth.py untuk login dan registrasi, routes.py untuk endpoint, services.py untuk layanan data manga), app.py sebagai titik masuk aplikasi, Dockerfile, requirements.txt, dan perpustakaan.sql.

project_manga/ berisi project Flutter (lib, assets, web, dan folder platform lainnya) beserta Dockerfile, nginx.conf, dan pubspec.yaml yang dipakai untuk membangun container frontend.

Folder utama container_manga/ memuat docker-compose.yml, .env, .env.example, README.md, dan Information.md. File .env tidak diunggah ke GitHub, sedangkan .env.example menjadi contoh isinya.

Gambar 14 Struktur root project

Penerapan database

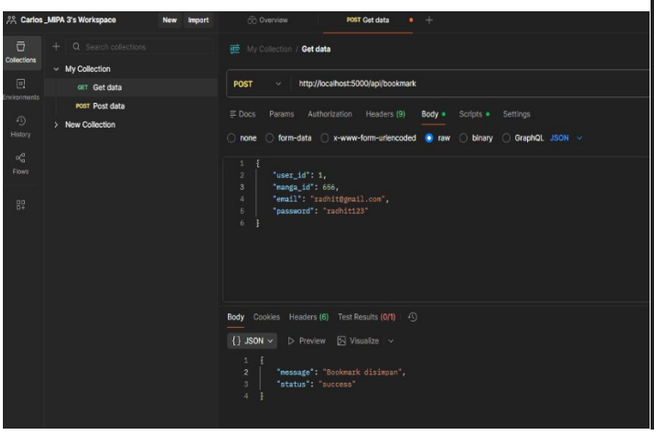
Tabel *users* (id, nama, email, password hash) dan *bookmarks* (id, user_id, manga_id, title, image_url, read_status, created_at) dihubungkan dengan foreign key. Bookmark ikut terhapus saat akun dihapus (ON DELETE CASCADE), dan kombinasi user_id dan manga_id bersifat unik.

Penerapan CRUD pada bookmark

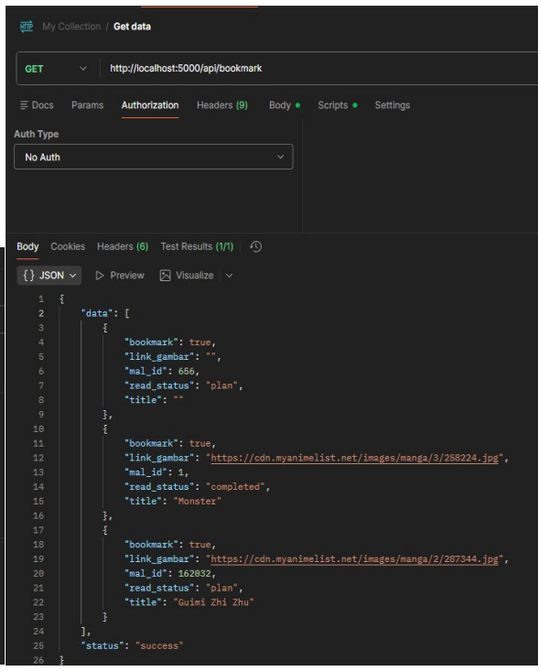
Operasi	Endpoint	Aksi di aplikasi
Create	POST /api/bookmark	Menyimpan manga ke rak buku

Operasi	Endpoint	Aksi di aplikasi
Read	GET /api/bookmark	Menampilkan koleksi dan status baca
Update	PUT /api/bookmark/<manga_id>	Mengubah status baca (Reading, Plan, atau Completed)
Delete	DELETE /api/bookmark/<manga_id>	Menghapus manga dari koleksi

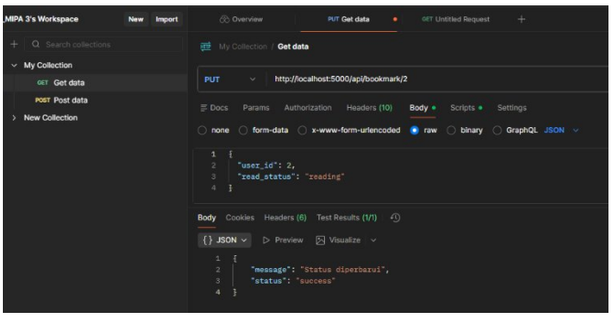
Hasil Penerapan CRUD sesuai urutan operasi:



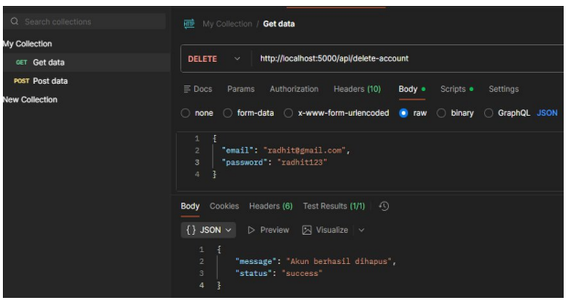
Gambar 15 Post Bookmark



Gambar 16 Get Bookmark



Gambar 17 Put Bookmark



Gambar 18 Delete akun

Sumber kode, repository, Docker Hub, dan referensi

Kode frontend dan backend ditulis sendiri oleh kelompok (rincian ada pada tabel status pengembang di bagian 1). Data manga diambil dari API publik Tenrai yang berbasis data MyAnimeList, lalu disimpan sementara (cache) di server selama 24 jam. Image manga_api dan manga_web dibangun dari Dockerfile milik kelompok dan dipublikasikan ke Docker Hub dengan nama mangatsu-api dan mangatsu-web, sedangkan image manga_db, manga_pma, serta image dasar Nginx dan Python adalah image resmi dari Docker Hub. Seluruh link repository, Docker Hub, dan referensi tercantum pada tabel berikut.

Sumber	Keterangan	Link
Repository GitHub	Kode lengkap: backend, frontend, Dockerfile, docker-compose.yml	https://github.com/carlos1739/mangatsu
Docker Hub: mangatsu-api	Image backend Flask buatan kelompok	https://hub.docker.com/r/carlossiregar/mangatsu-api
Docker Hub: mangatsu-web	Image frontend Flutter buatan kelompok	https://hub.docker.com/r/carlossiregar/mangatsu-web
Docker Hub: MySQL	mysql:8.0 (container manga_db)	https://hub.docker.com/_/mysql
Docker Hub: phpMyAdmin	phpmyadmin/phpmyadmin (container manga_pma)	https://hub.docker.com/r/phpmyadmin/phpmyadmin
Docker Hub: Nginx	nginx:alpine (image dasar manga_web)	https://hub.docker.com/_/nginx
Docker Hub: Python	python:3.12-slim (image dasar manga_api)	https://hub.docker.com/_/python
API data manga	Tenrai API (berbasis MyAnimeList)	https://api.tenrai.org
Dokumentasi Docker	Docker dan Docker Compose	https://docs.docker.com/compose/
Dokumentasi Flutter	Framework frontend	https://docs.flutter.dev
Dokumentasi Flask	Framework backend	https://flask.palletsprojects.com
Dokumentasi MySQL	Database	https://dev.mysql.com/doc/
Dokumentasi Nginx	Web server	https://nginx.org/en/docs/

Detail image di Docker Hub

Dua image buatan kelompok dipublikasikan di akun Docker Hub carlossiregar: mangatsu-api untuk backend dan mangatsu-web untuk frontend. Kedua image dibangun dari Dockerfile yang kami tulis sendiri, sehingga isinya sama dengan container manga_api dan manga_web yang dijalankan lewat Docker Compose. Image untuk manga_db (mysql:8.0) dan manga_pma (phpmyadmin) tidak kami upload karena memakai image resmi langsung dari Docker Hub. Rincian kedua image kami adalah sebagai berikut.

Keterangan	mangatsu-api	mangatsu-web
Repository	carlossiregar/mangatsu-api	carlossiregar/mangatsu-web
Isi image	Backend Flask yang dijalankan dengan Gunicorn	Hasil build Flutter Web yang disajikan oleh Nginx
Image dasar	python:3.12-slim	nginx:alpine
Dibangun dari	backend_manga/Dockerfile	project_manga/Dockerfile
Port di dalam container	5000	80
Dipakai sebagai container	manga_api (port 5000:5000)	manga_web (port 8080:80)
Tag	sha-<kode commit>, latest, 1.1	sha-<kode commit>, latest, 1.1
Perintah pull	`docker pull carlossiregar/mangatsu-`	`docker pull carlossiregar/mangatsu-`

Keterangan	mangatsu-api	mangatsu-web
	`api:latest`	`web:latest`

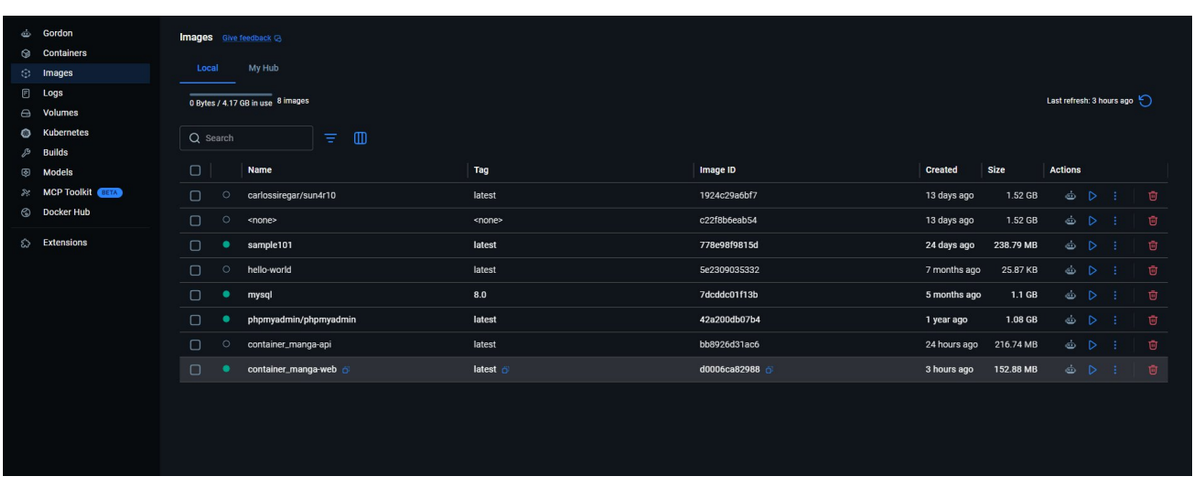
Tag sha-<kode commit> dibuat pada setiap build, jadi setiap versi image bisa dilacak ke commit tertentu di GitHub. Tag latest dan 1.1 hanya diberikan jika build berasal dari branch utama, sebagai versi yang dianggap stabil. Image tidak kami upload secara manual: proses build dan push berjalan otomatis lewat GitHub Actions memakai access token Docker Hub, seperti yang dijelaskan pada bagian Implementasi CI/CD di akhir laporan.

4. File Pendukung dan Aplikasi Tambahan

File / aplikasi	Kegunaan
Dockerfile (backend dan frontend)	Resep pembuatan image backend dan frontend.
docker-compose.yml	Mendefinisikan dan menjalankan empat container sekaligus.
.env	Menyimpan nama database, user, dan password (tidak diunggah ke GitHub).
.env.example	Contoh isi file .env tanpa data rahasia, untuk dibagikan di GitHub.
README.md	Petunjuk singkat project di repository GitHub.
perpustakaan.sql	Skema dan data awal database, dijalankan otomatis saat container database pertama dibuat.
nginx.conf	Konfigurasi Nginx untuk menyajikan Flutter Web.
requirements.txt	Daftar library Python backend.
Docker Desktop	Menjalankan container di Windows.
Flutter SDK, VS Code	Membangun frontend dan menyunting kode.
phpMyAdmin	Memeriksa dan mengelola isi database.

Tampilan container

Tampilan container yang berjalan di Docker Desktop: mysql (manga_db), phpmyadmin (manga_pma), container_manga-api (manga_api), dan container_manga-web (manga_web), seperti pada gambar berikut.



Gambar 19 Tampilan Docker Desktop

5. Link Video Dokumentasi

Video dokumentasi kami bagi menjadi dua, yaitu video penerapan container dan video proses CI/CD. Tiap video punya tautan sendiri pada tabel berikut. Tautan folder SharePoint tetap kami cantumkan sebagai arsip seluruh file video.

No	Video	Isi video	Tautan langsung
1	Video 1: Container	Penyusunan file Docker, build, menjalankan container, sampai demo CRUD.	Video 1
2	Video 2: CI/CD	Proses CI/CD dengan GitHub Actions.	Video 2
3	Folder arsip	Folder SharePoint yang berisi file video.	Folder video Mangatsu (SharePoint)

Izin akses ketiga tautan diatur ke mode lihat saja, supaya dosen bisa membuka video langsung tanpa harus meminta izin terlebih dulu.

6. Originalitas

Mangatsu dibuat bukan hanya untuk mencari manga, tetapi juga untuk membantu pengguna mengatur bacaan mereka. Setelah menemukan manga melalui pencarian atau genre, pengguna bisa melihat detail dan chapter yang tersedia, lalu membacanya langsung dari aplikasi. Manga yang ingin dibaca dapat disimpan ke bookmark dengan status Plan, Reading, atau Completed, atau ditambahkan ke daftar favorit. Saat membaca, progres terakhir juga tersimpan sehingga pengguna bisa melanjutkan dari halaman yang sama. Selain itu, pengguna dapat memberi ulasan dan melihat bagian Trending yang dibuat berdasarkan aktivitas membaca di Mangatsu.

Agar fitur-fitur tersebut berjalan, Mangatsu menggunakan Flutter Web sebagai tampilan aplikasi, Flask sebagai backend, dan MySQL untuk menyimpan data pengguna serta aktivitas membaca. Informasi manga dan chapter diambil dari MangaDex API. Data katalog disimpan sementara di cache backend hingga 24 jam, sedangkan gambar halaman chapter dimuat langsung dari server MangaDex. Seluruh layanan aplikasi dijalankan bersama menggunakan Docker Compose.

Update Perubahan

Bagian 1 sampai 6 menjelaskan aplikasi pada versi awal, yaitu saat sumber katalog masih Tenrai API, container dibangun bertahap per bagian, dan seluruh screenshot di atas diambil dari versi tersebut. Bagian ini mencatat perubahan sesudahnya, dengan penomoran sendiri yang dimulai dari 1: ringkasan, latar belakang perubahan API, integrasi MangaDex, cara build dan menjalankan dengan Docker Compose, fitur baru, database terbaru, dan seluruh endpoint API. Empat container dan port-nya tidak berubah.

1. Ringkasan perubahan

Aspek	Versi awal (bagian 1-6)	Setelah update
Build dan menjalankan	Dikerjakan bertahap per bagian: backend, lalu build web dan container frontend	Satu perintah Docker Compose memproses semua service
Sumber katalog	Tenrai API (api.tenrai.org), berbasis data MyAnimeList; hanya metadata manga	MangaDex API (api.mangadex.org); metadata, daftar chapter, dan halaman chapter
ID manga	ID angka dari sumber lama	UUID MangaDex; kolom manga_id bertipe CHAR(36)
Membaca chapter	Belum tersedia	Daftar chapter bahasa Indonesia dan Inggris serta halaman chapter dari MangaDex, ditampilkan di reader
Cache	Data katalog disimpan sementara di server selama 24 jam	Hanya data katalog yang di-cache di memori backend (maksimal 24 jam); halaman chapter tidak di-cache dan tidak disimpan di database
Genre	Genre dihitung dari data katalog	Jumlah manga per genre memakai total hasil MangaDex; genre dengan total nol tidak ditampilkan
Tabel database	2 tabel: users dan bookmarks	6 tabel: users, bookmarks, favorit, reviews, reading_progress, reading_history
Favorit	Belum tersedia	Tombol favorit di halaman detail; data disimpan pada tabel favorit
Endpoint	Bookmark (CRUD) dan akun	Ditambah katalog, chapter dan reader, Trending, genre, ulasan, favorit, dan progres baca
Container dan port	manga_db 3306 (internal), manga_api 5000:5000, manga_web 8080:80, manga_pma 8082:80	Tidak berubah

2. Latar belakang perubahan API

Pada versi awal, Mangatsu memakai Tenrai API sebagai sumber data. API tersebut hanya menyediakan metadata manga, yaitu informasi katalog seperti judul, sampul, penulis, dan genre, tanpa data chapter maupun halaman baca. Akibatnya, aplikasi versi awal hanya dapat menjelajahi katalog dan menyimpan bookmark, sedangkan pengguna tidak dapat membaca cerita manga dari dalam aplikasi. Oleh karena itu, kami ingin membuat layar detail chapter, yaitu daftar chapter pada halaman detail manga dan reader untuk membaca halaman chapter. Kebutuhan itu tidak dapat dipenuhi hanya

dengan metadata, sehingga sumber data dipindah ke MangaDex API yang menyediakan metadata sekaligus daftar chapter dan halaman chapter.

Kebutuhan data	Tenrai API (versi awal)	MangaDex API (setelah update)
Metadata manga (judul, sampul, penulis, genre)	Tersedia	Tersedia
Daftar chapter	Tidak tersedia	Tersedia (bahasa Indonesia dan Inggris)
Halaman baca chapter	Tidak tersedia	Tersedia (URL halaman gambar)
Layar detail chapter dan reader	Tidak dapat dibuat	Dapat dibuat

3. Perubahan integrasi dari Tenrai API ke MangaDex API

Backend (manga_api) sekarang memanggil MangaDex API untuk katalog, pencarian, detail manga, genre, daftar chapter, dan halaman chapter. Detail manga menampilkan judul, sampul, penulis, status, sinopsis, tahun rilis, dan genre. Rating yang tampil berasal dari ulasan pengguna Mangatsu yang tersimpan di MySQL, bukan rating MangaDex, karena rating MangaDex belum disediakan oleh integrasi saat ini.

Karena MangaDex beralih menggunakan UUID, bookmark lama yang berbasis ID angka tidak dapat digunakan kembali. Untuk mengatasi hal ini, skrip migrasi backend_manga/database/migrate_mangadex.sql akan menghapus data bookmark lama tersebut dengan tetap mempertahankan akun pengguna yang ada.

Pada database yang sudah berjalan, proses migrasi ini hanya perlu dijalankan satu kali. Setelah skrip migrasi selesai dieksekusi, bangun ulang (rebuild) API dan layanan web menggunakan perintah berikut:

```
docker compose up --build -d
```

Diagram berikut menunjukkan arsitektur setelah update. Container dan port sama seperti sebelumnya; yang berubah adalah sumber data eksternal dan alur baca halaman chapter.

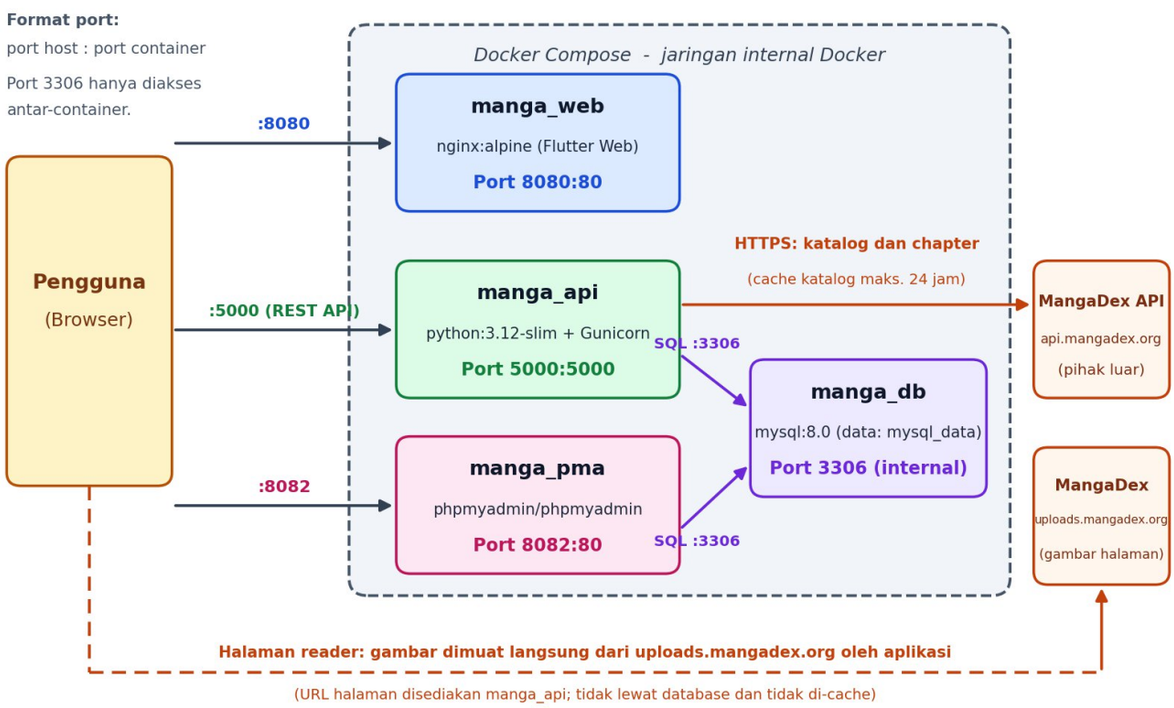


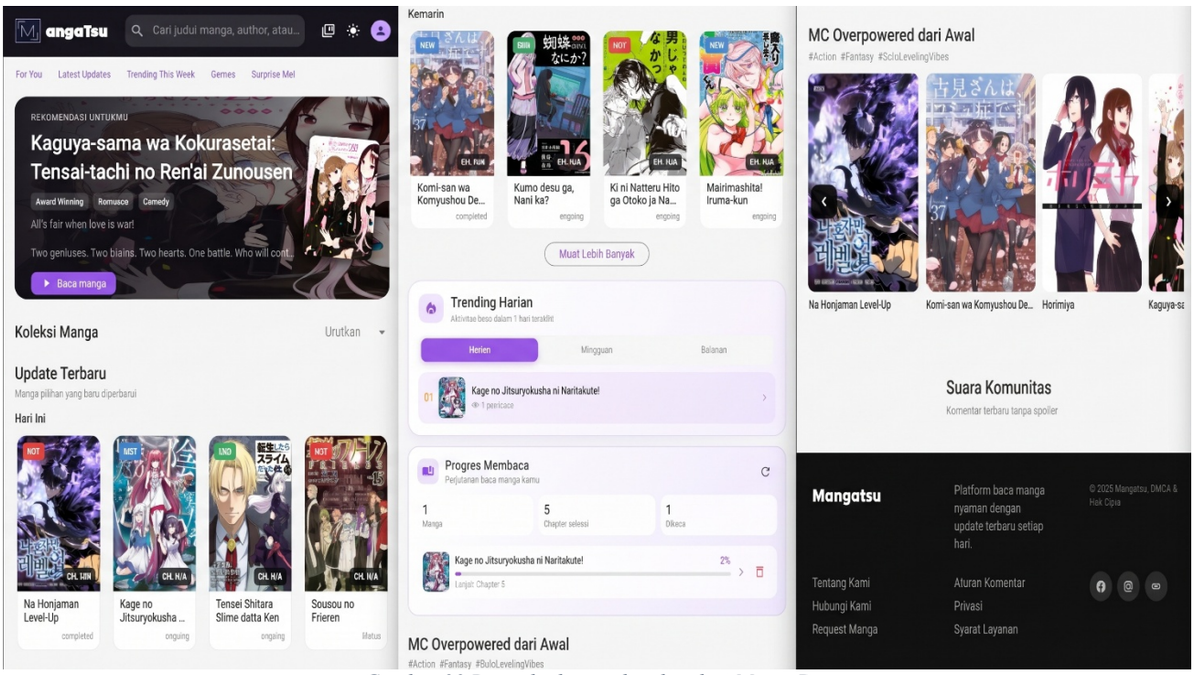
Diagram 2 Arsitektur, port, dan alur data setelah update (MangaDex)

Alur data

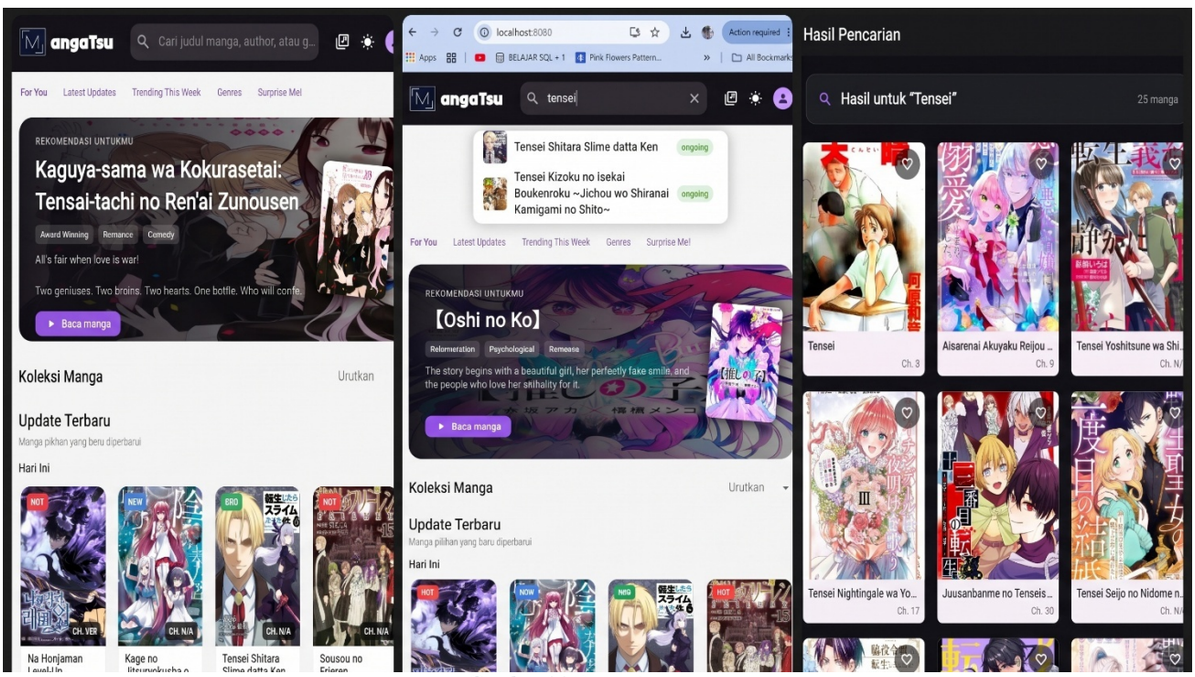
Aplikasi di browser memanggil manga_api untuk katalog, pencarian, detail manga, genre, daftar chapter, dan URL halaman chapter. manga_api mengambil data tersebut dari MangaDex API; hanya data katalog yang disimpan sementara di cache memori backend maksimal 24 jam. Gambar halaman chapter dimuat langsung oleh aplikasi dari uploads.mangadex.org menggunakan URL yang disediakan manga_api, tidak melewati database, dan tidak disimpan di database Mangatsu.

Referensi tambahan

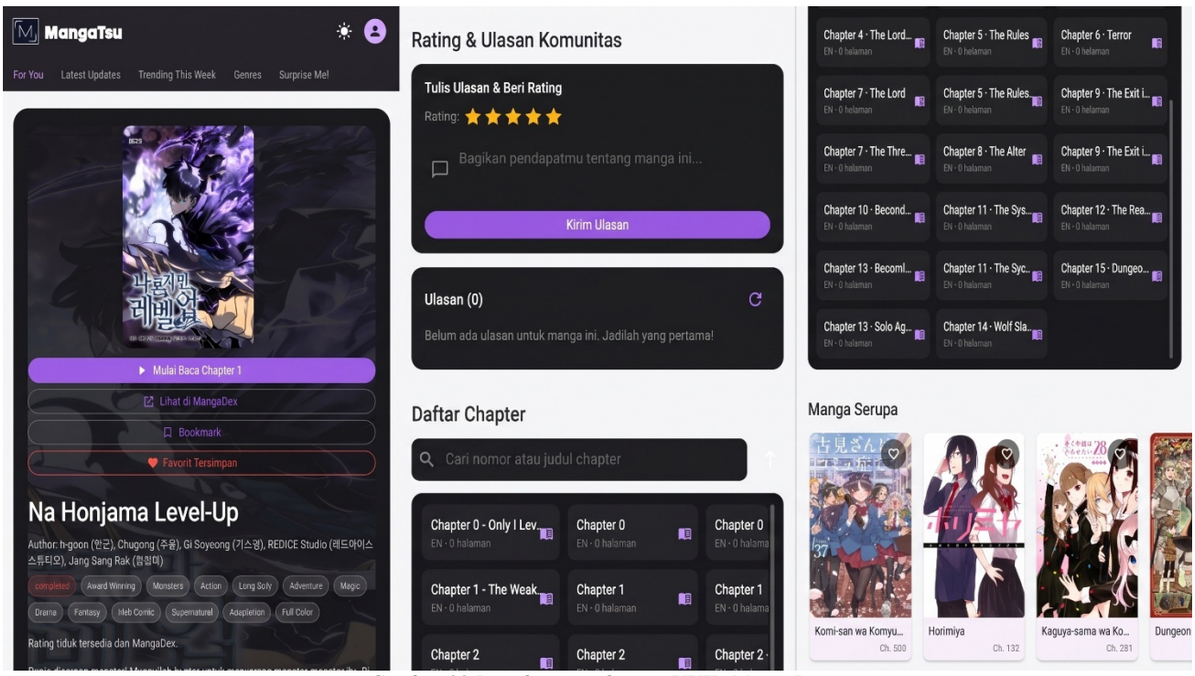
Dokumentasi MangaDex API: https://api.mangadex.org/docs/. Tenrai API tidak lagi dipakai, sehingga pada tabel status pengembang di bagian 1 laporan awal, data katalog kini bersumber dari MangaDex API.



Gambar 20 Beranda dengan katalog dari MangaDex



Gambar 21 Pencarian manga



Gambar 22 Detail manga dengan UUID MangaDex

4. Build dan menjalankan dengan Docker Compose

Pada versi awal, image backend dan web dibangun bertahap per bagian (lihat Gambar 1 sampai 3). Pada update ini seluruh service sudah didefinisikan dalam satu file docker-compose.yml, sehingga satu perintah Compose memproses semua service yang ada. Perbedaannya dengan cara manual per folder adalah sebagai berikut.

Aspek	Manual per folder	Docker Compose
Perintah	`docker build -t api ./backend_manga`, `docker build -t web ./project_manga`, lalu menjalankan container satu per satu	`docker compose up --build -d`
Definisi service	Tersebar pada perintah yang diketik terpisah	Satu file docker-compose.yml berisi seluruh service, port, volume, dan variabel environment
Proses build	Satu per satu secara berurutan	Service yang saling independen umumnya dapat dibangun secara paralel
Database	Dijalankan sendiri dari image MySQL	Memakai image MySQL langsung (tanpa build), dibuat bersamaan dengan service lain
Urutan start	Diatur manual oleh pengguna	Database memiliki health check; API dan phpMyAdmin menunggu database sehat
Perawatan	Perintah harus diingat dan diulang untuk tiap bagian	Satu orkestrasi yang lebih rapi dan mudah diulang di komputer lain

Keterangan opsi -t.

Pada perintah `docker build -t api ./backend_manga`, opsi -t adalah singkatan dari --tag, yaitu memberi nama (tag) pada image yang dibuat. Tanpa -t, image hanya memiliki ID acak yang sulit diingat dan dipanggil. Dengan -t api, image diberi nama api; format lengkapnya nama:tag, misalnya api:latest atau api:1.0, dan bila tag tidak ditulis Docker memakai latest secara otomatis. Nama inilah yang dipakai saat menjalankan container, misalnya `docker run -p 5000:5000 api`. Bagian ./backend_manga di akhir perintah adalah build context, yaitu folder yang berisi Dockerfile.

Pada Docker Compose nama image sudah ditentukan oleh docker-compose.yml, sehingga opsi -t tidak perlu diketik lagi.

Pada project ini terdapat backend_manga/Dockerfile untuk API dan project_manga/Dockerfile untuk web, sedangkan database dan phpMyAdmin memakai image langsung dari Docker Hub. Saat docker compose up --build -d dijalankan, Docker membangun image API, membangun image web, lalu menyiapkan container database dan phpMyAdmin dalam satu alur kerja yang terkoordinasi. Service api dan web memiliki Dockerfile masing-masing sehingga imagenya tetap dapat dibangun terpisah (misalnya docker compose build api) tanpa meninggalkan file Compose yang sama.

Langkah menjalankan setelah update. Image web menyalin hasil build Flutter Web, sehingga build Flutter dilakukan lebih dulu dari folder project_manga, kemudian seluruh container dijalankan dari folder utama:

```
cd project_manga
flutter pub get
flutter build web --release
cd ..
docker compose up --build -d
docker compose ps
```

Perintah lain yang dipakai: `docker compose logs -f api` untuk melihat log backend, dan `docker compose down` untuk menghentikan container dengan mempertahankan data. Perintah `docker compose down -v` menghapus volume mysql_data beserta seluruh data database lokal, sehingga hanya dipakai bila memang ingin menghapus data. Alamat yang dipakai ada tiga: http://localhost:8080 untuk aplikasi, http://localhost:5000/api/health untuk health check API, dan http://localhost:8082 untuk phpMyAdmin.

Alur langkah build dan menjalankan aplikasi setelah update dirangkum pada Diagram 3. Jika salah satu container tidak berstatus Up pada langkah pemeriksaan, pengguna membaca log backend, memperbaiki masalahnya, lalu mengulang perintah Docker Compose.

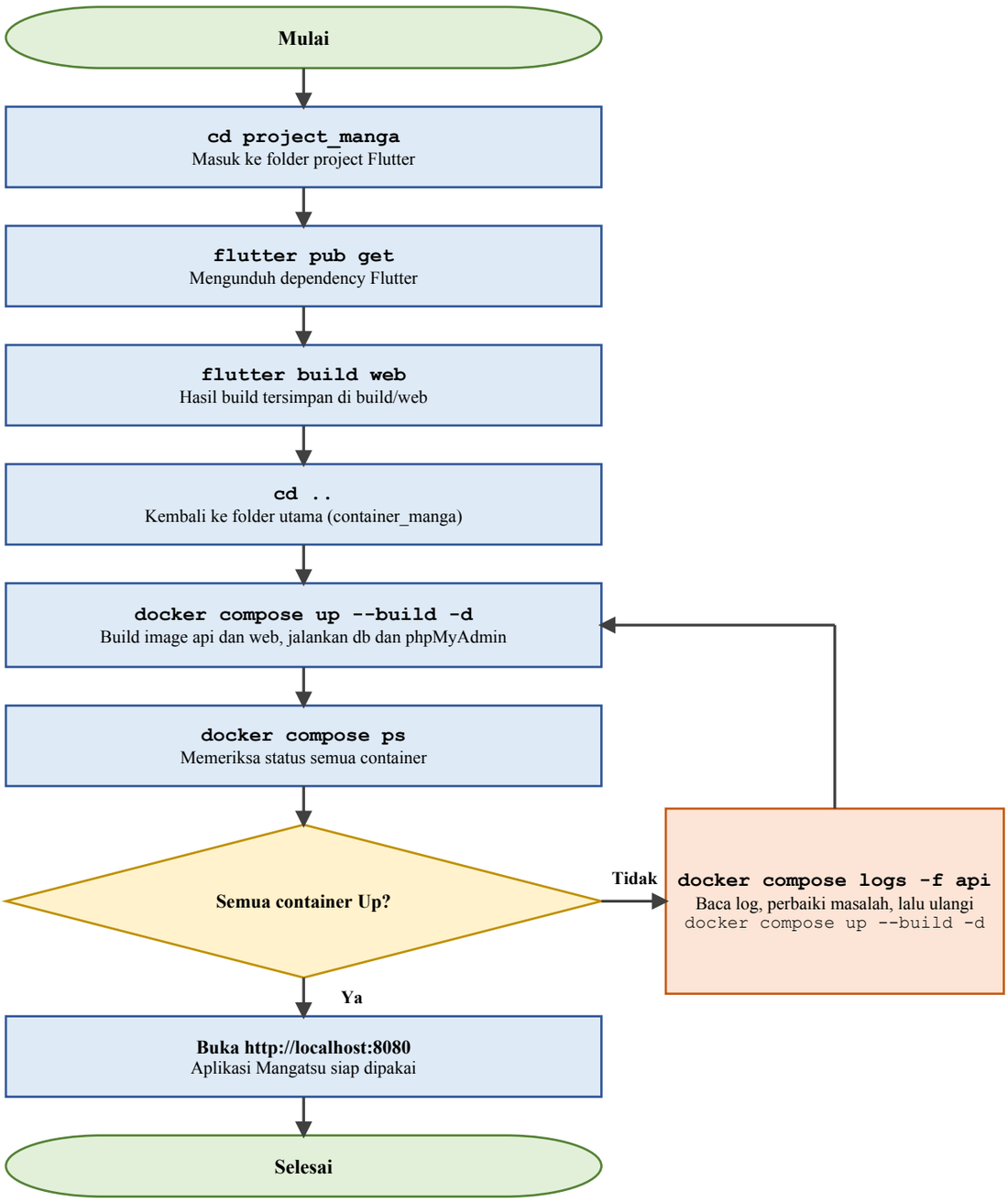


Diagram 3 Alur langkah build dan menjalankan aplikasi dengan Docker Compose

```
PS D:\container_manga> docker compose up --build -d
[+] Building 7.1s (23/23) FINISHED
 => [internal] load local bake definitions
 => => reading from stdin 9638
 => [web internal] load build definition from Dockerfile
 => => transferring dockerfile: 1538
 => [api internal] load build definition from Dockerfile
 => => transferring dockerfile: 2988
 => [web internal] load metadata for docker.io/library/nginx:alpine
 => [api internal] load metadata for docker.io/library/python:3.12-slim
 => [auth] library/python:pull token for registry-1.docker.io
 => [auth] library/nginx:pull token for registry-1.docker.io
 => [web internal] load .dockerignore
 => => transferring context: 2B
 => [api internal] load .dockerignore
 => [web internal] load build context
 => => transferring context: 3.34KB
 => [api internal] load build context
 => => transferring context: 1.04KB
 => [api 1/5] FROM docker.io/library/python:3.12-slim@sha256:aa943ce411423c7582288f18707ff66
 => => resolve docker.io/library/python:3.12-slim@sha256:aa943ce411423c7582288f18707ff66oddc
 => CACHED [web 2/3] COPY build/web /usr/share/nginx/html
 => CACHED [web 2/3] COPY build/web /usr/share/nginx/html
 => CACHED [web 2/3] COPY nginx.conf /etc/nginx/conf.d/default.conf
 => [web] exporting to image
 => => transferring load layers
 => => transferringload .dockerignore
 => [api internal] load build context
 => => transferring context: 2B
 => [api internal] load .dockerignore
 => => transferring context: 70B
 => [web 1/3] FROM docker.io/library/nginx:alpine@sha256:785335ea65dbabba6f021acc0f3be91b8f
 => => resolve docker.io/library/nginx:alpine@sha256:788335ea65dbabba6f021acc0f3be91b0fef764
 => [web internal] load build context
 => => transferring context: 3.34KB
 => [api internal] load build context
 => => transferring context: 1.04KB
 => [api 1/5] FROM docker.io/library/python:3.12-slim@sha256:aa943ce411422c7582288f18707ff66
 => => resoive docker.io/library/python:3.12-slim@sha256:aa943ce411423c7582288f18707ff66oddc
 => CACHED [web 2/3] COPY build/web /usr/share/nginx/html
 => CACHED [web 3/3] COPY nginx.conf /etc/nginx/conf.d/default.conf
 => [web] exporting to image
 => [web] exporting to image
 => => exporting layers
 => => exporting manifest sha256:159eb6dacbf7bb493d7dcd0d33eeecfec9e44f82b93e2bd712de9fb0c
 => => exporting config sha256:46195e48a33ff177809364238fh15829382f9980cabb153
 => => exporting attestation manifest sha256:d78e10176e444c46ddcdddb767eb2222ebaf777331f62
 => => exporting manifest list sha256:7635082858348c40d3a92d9247hfb2e86dd1c9e9de5c5507f534
 => => naming to docker.io/library/container_manga-web:latest
 => => uxporting layers
 => CACHED [api 2/5] WORKDIR /app
 => CACHED [api 4/5] RUNY install -- no-cache-dir -r requirements.txt
 => => exporting layers
 => CACHED [api 2/5] COPY requements.txt .
 => CACHED [api 4/5] RUN .. --no-cache-dir -r requirements.txt
 => [api] exporting to image
 => => exporting layers
 => => exporting manifest sha256:09f6eaaba4be011aadf465764895966d7829228051c2111eb5c33333
 => => exporting manifest list sha256:002acb1122586feee5a5177f333a15b15cb4dd8755840e
 => => exporting config sha256:5faac1ee0348db356eb53f141670895766b5fd182e7ad26d887
 => => exporting attestation manifest sha256:39a96441ebefaa1a3e2e9597321fe438e7280a6dbe97
 => => exporting manifest list sha256:e0388398d88ae8b3eeefe2fe141859ffc39ec70f16900ab89eb
 => => exporting manifest list sha256:e0388398d88ae8b3eeefe2fe141858ac5fde70f1963bffb910c
 => => naming to docker.io/library/container_manga-api:latest
 => => unpacking to docker.io/library/container_manga-api:latest
 => [web] resolving provenance for metadata file
 => [api] resolving provenance for metadata file
[+]  8/8/8
 ✓ Image container_manga-api          Built
 ✓ Image container_manga-web          Built
 ✓ Network container_manga_default    Created
 ✓ Volume container_manga_mysql_data  Created
 ✓ Container manga_db                 Healthy
 ✓ Container manga_api                Started
 ✓ Container manga_pma                Started
```

Gambar 23 Proses `docker compose up --build -d`

```
PS D:\container_manga> docker compose ps
NAME        IMAGE                  COMMAND                  SERVICE      CREATED         STATUS                   PORTS
manga_api   container_manga-api    "gunicorn -w 2 -b 0.…"   api          3 minutes ago   Up 2 minutes             0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp
manga_db    mysql:8.0              "docker-entrypoint.s…"   db           3 minutes ago   Up 3 minutes (healthy)   3306/tcp, 33060/tcp
manga_pma   phpmyadmin/phpmyadmin  "/docker-entrypoint.…"   phpmyadmin   3 minutes ago   Up 2 minutes             0.0.0.0:8082->80/tcp, [::]:8082->80/tcp
manga_web   container_manga-web    "/docker-entrypoint.…"   web          3 minutes ago   Up 2 minutes             0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
PS D:\container_manga>
```

Gambar 24 Status container dengan `docker compose ps`

```
PS D:\container_manga\project_manga> flutter build web
Resolving dependencies...
Downloading packages...
  async 2.12.0 (2.13.1 available)
  clock 1.1.2 (1.1.3 available)
  cupertino_icons 1.0.8 (2.0.0 available)
  flutter_lints 5.0.0 (6.0.0 available)
  lints 5.1.1 (6.1.0 available)
  material_color_utilities 0.13.0 (0.13.1 available)
  source_span 1.10.1 (1.10.2 available)
  stack_trace 1.12.1 (1.12.2 available)
  test_api 0.7.12 (0.7.15 available)
  vector_math 2.4.3 (2.4.4 available)
  vm_service 14.3.1 (15.4.0 available)
  web 1.1.1 (1.1.2 available)
Got dependencies!
12 packages have newer versions incompatible with dependency constraints.
Try `flutter pub outdated` for more information.
Wasm dry run succeeded. Consider building and testing your application with the `--wasm` flag. See
Use --no-wasm-dry-run to disable these warnings.
Font asset "CupertinoIcons.ttf" was tree-shaken, reducing it from 257628 to 1472 bytes (99.4% reduc
flag when building your app.
Font asset "MaterialIcons-Regular.otf" was tree-shaken, reducing it from 1645184 to 14408 bytes (99
--no-tree-shake-icons flag when building your app.
Compiling lib\main.dart for the Web...                             178.2s
√ Built build\web
PS D:\container_manga\project_manga>
```

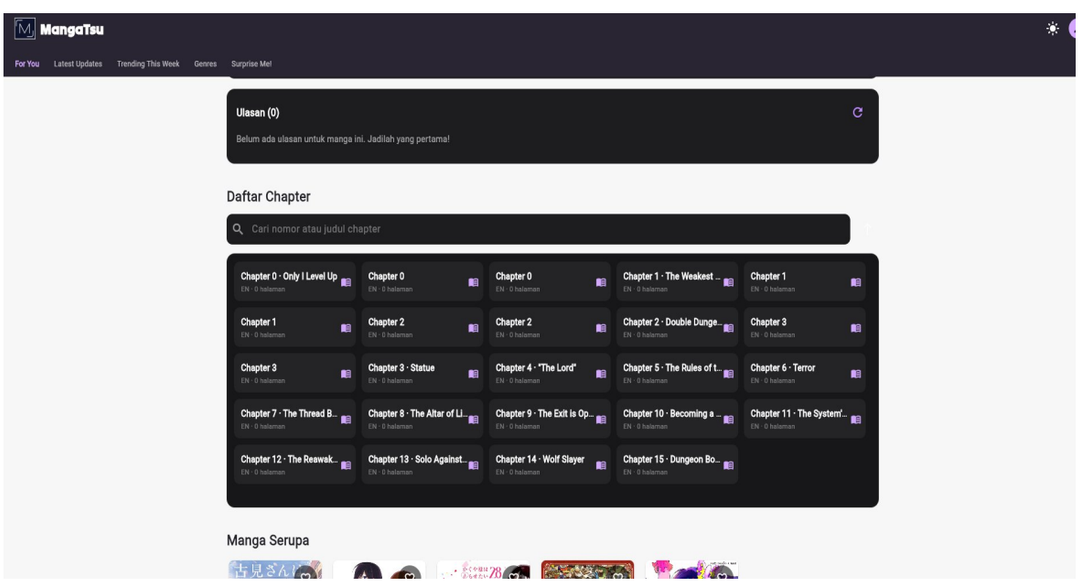
Gambar 25 Flutter build web

5. Fitur baru

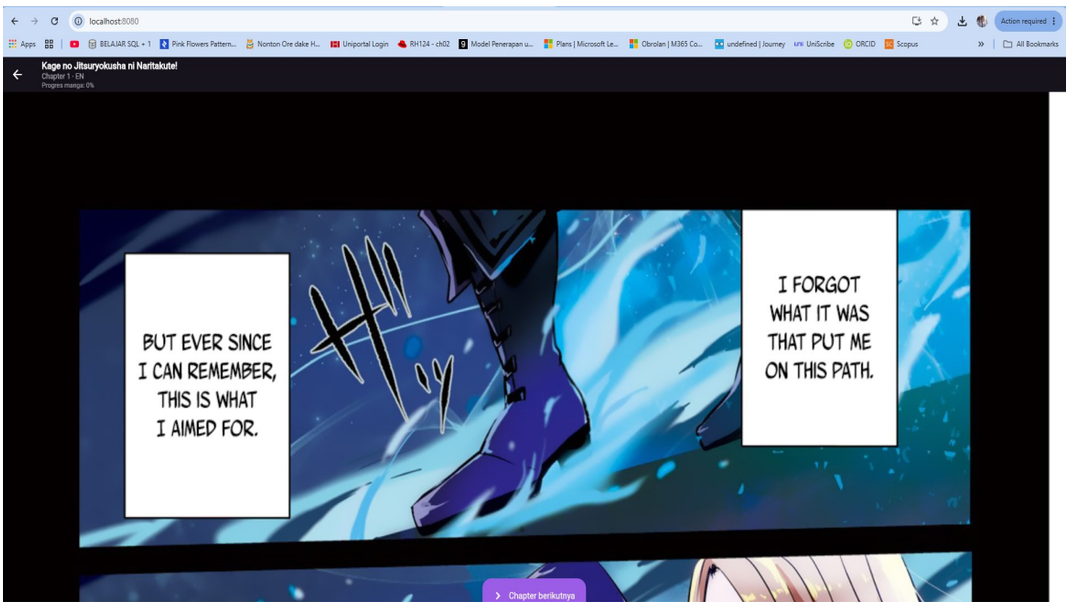
Fitur bookmark, registrasi, login, dan hapus akun tetap seperti pada versi awal. Fitur yang ditambahkan pada update ini dibangun kelompok pada backend Flask, frontend Flutter, dan database MySQL, dengan MangaDex hanya sebagai sumber data katalog dan chapter.

5.1 Reader chapter

Dari halaman detail, pengguna membuka daftar chapter bahasa Indonesia dan Inggris, lalu membaca halaman chapter di reader. Endpoint: GET /api/manga/{manga_id}/chapters dan GET /api/chapter/{chapter_id}/pages. Ketersediaan chapter bergantung pada unggahan di MangaDex dan berbeda-beda untuk setiap manga, sehingga tidak semua manga memiliki chapter berbahasa Indonesia atau Chapter 1. Halaman dibaca langsung dari server MangaDex dan tidak disimpan di database Mangatsu.



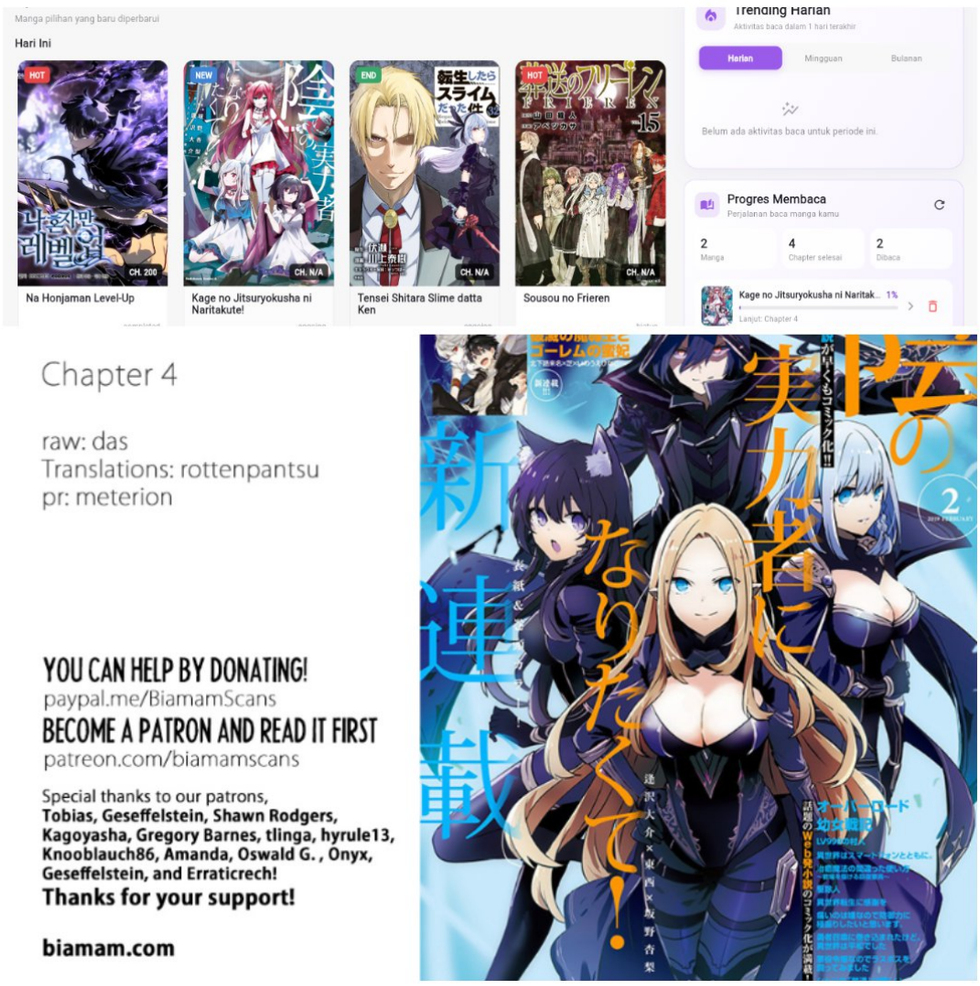
Gambar 26 Daftar chapter



Gambar 27 Halaman reader

5.2 Progres dan riwayat baca

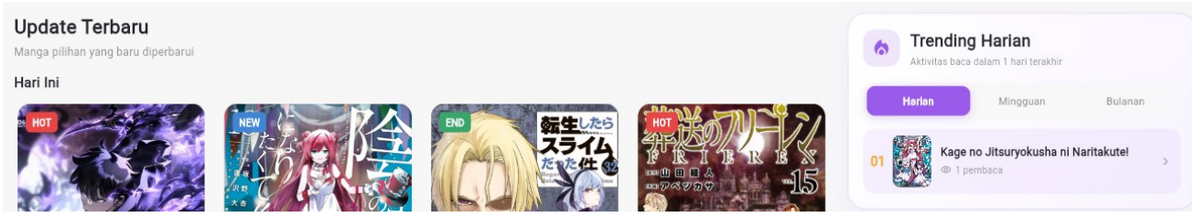
Reader menyimpan halaman terakhir yang dibaca pada setiap chapter. Saat pembaca mencapai akhir chapter, chapter ditandai selesai. Data ini dipakai untuk melanjutkan membaca dan menampilkan persentase progres per manga. Endpoint: GET dan DELETE /api/reading/progress/{manga_id}, POST /api/reading/progress, dan GET /api/reading/summary. Data disimpan pada tabel reading_progress dan reading_history.



Gambar 28 Progres dan lanjut membaca

5.3 Ranking Trending

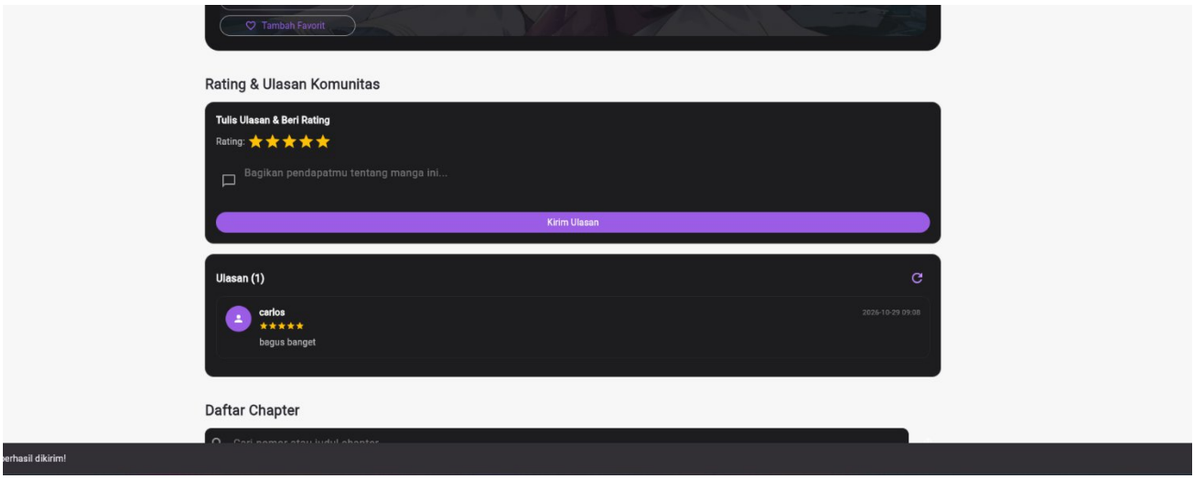
Beranda menampilkan ranking Trending harian, mingguan, dan bulanan berdasarkan jumlah pembaca unik dalam 1, 7, dan 30 hari terakhir. Ranking dihitung langsung dari tabel reading_history, jadi berasal dari aktivitas baca pengguna Mangatsu, bukan angka popularitas MangaDex; hasil tiap periode dapat sama bila aktivitas dalam rentang waktunya sama. Endpoint: GET /api/manga/trending?period=daily, weekly, atau monthly.



Gambar 29 Ranking Trending

5.4 Ulasan dan rating

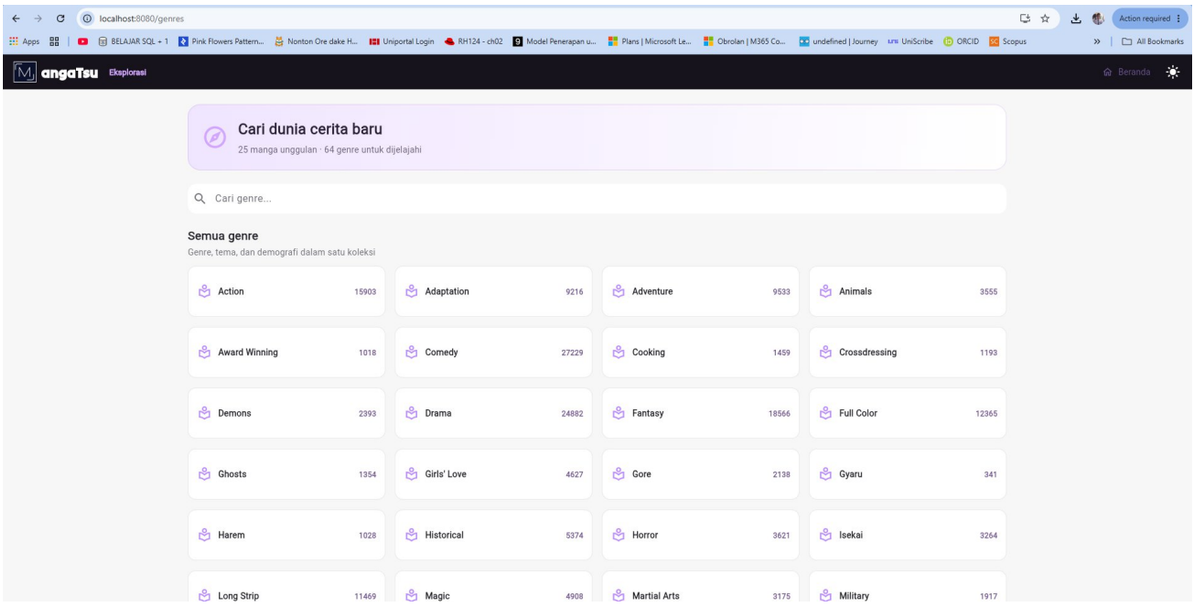
Halaman detail manga menampilkan ulasan dan rating pengguna, dan pengguna dapat mengirim ulasannya sendiri. Endpoint: GET dan POST /api/manga/{manga_id}/reviews. Data disimpan pada tabel reviews.



Gambar 30 Ulasan dan rating

5.5 Eksplorasi genre

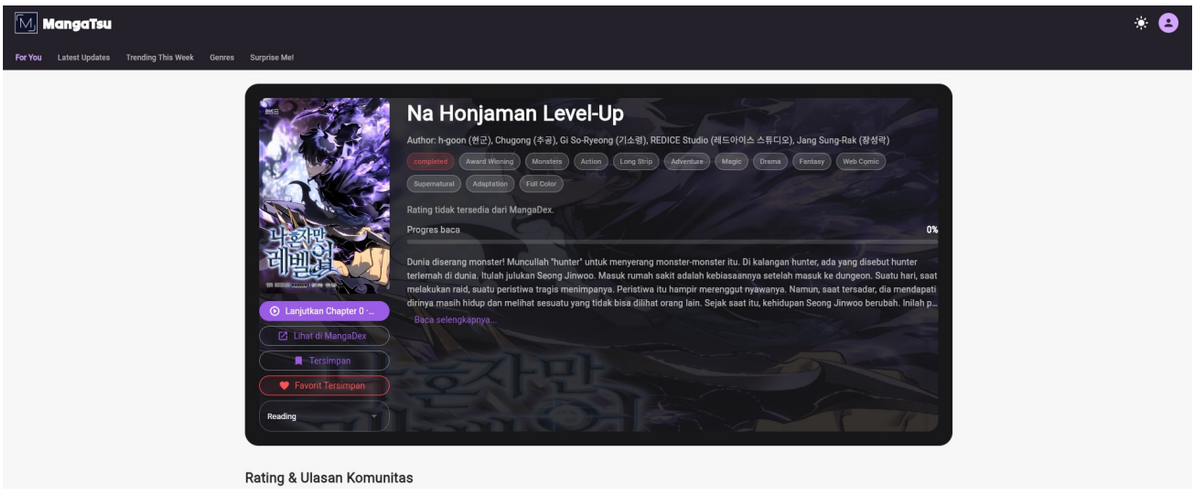
Halaman genre menampilkan jumlah manga per genre berdasarkan total hasil MangaDex, menyembunyikan genre dengan total nol, dan menampilkan manga dengan paginasi 25 per halaman. Endpoint: GET /api/manga/genre-counts?genres= dan GET /api/manga/genre/{genre_name}?page=.



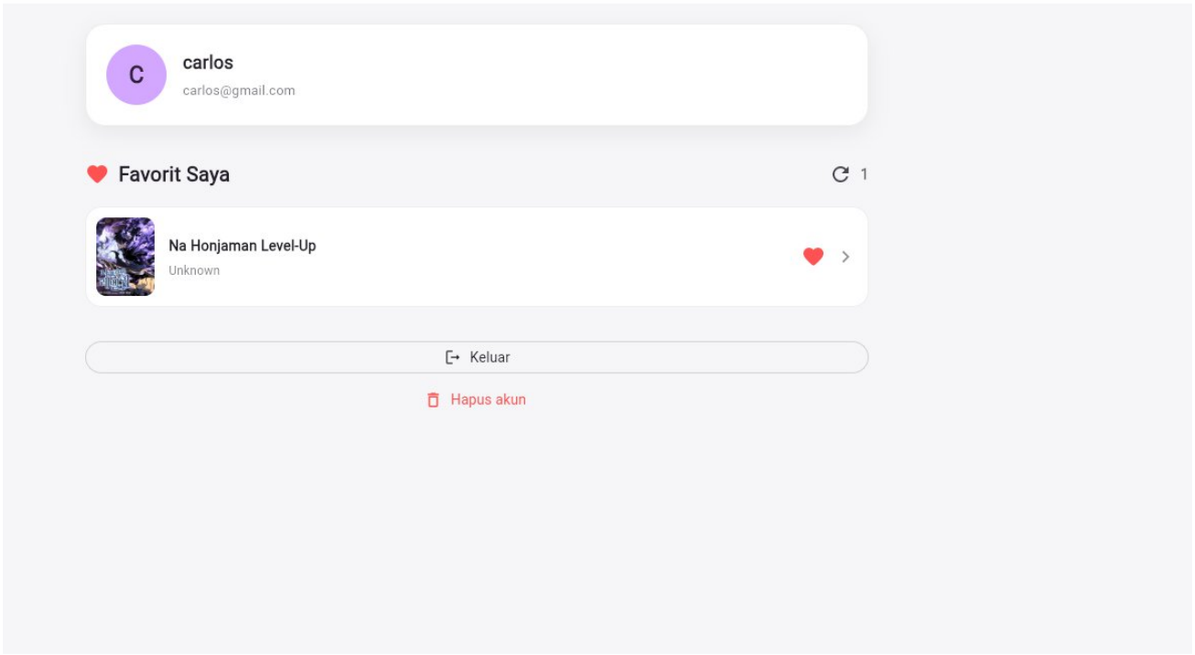
Gambar 31 Eksplorasi genre

5.6 Favorit

Selain bookmark dengan status baca, pengguna yang sudah login dapat menandai manga sebagai favorit dari halaman detail. Tombol favorit berada di bawah tombol Bookmark: berlabel Tambah Favorit saat manga belum difavoritkan, dan berlabel Favorit Tersimpan dengan ikon hati merah saat sudah; menekannya lagi menghapus favorit. Aplikasi memeriksa status favorit setiap halaman detail dibuka, dan pengguna yang belum login diminta masuk terlebih dahulu. Berbeda dari bookmark, favorit tidak memiliki status baca. Data disimpan pada tabel favorit (manga_id, judul, dan URL sampul per pengguna), dan satu manga hanya dapat difavoritkan satu kali oleh setiap pengguna. Endpoint: POST dan GET /api/favorites, serta GET dan DELETE /api/favorites/{manga_id}. Daftar favorit pengguna sudah tersedia di backend melalui GET /api/favorites; halaman khusus untuk menampilkannya di aplikasi belum ada pada versi ini.



Gambar 32 Tombol Tambah Favorit



Gambar 33 Tombol Favorit Tersimpan

6. Database terbaru

Database perpustakaan kini memiliki enam tabel yang dibuat otomatis dari perpustakaan.sql saat volume MySQL masih kosong (skrip tidak dijalankan ulang pada volume yang sudah terisi, sehingga database lama memakai skrip migrasi pada bagian 3 di Update Perubahan ini). Tabel bookmarks, favorit, reviews, reading_progress, dan reading_history dihubungkan ke users dengan foreign key ON DELETE CASCADE, sehingga data terkait ikut terhapus saat akun dihapus.

Tabel	Kolom utama	Aturan dan fungsi
users	id, nama, email, password	Email unik; password disimpan sebagai hash (Werkzeug)
bookmarks	id, user_id, manga_id, title, image_url, read_status, created_at	Status default plan; unik per (user_id, manga_id); manga_id CHAR(36)
favorit	id, user_id, manga_id, title, image_url, created_at	Unik per (user_id, manga_id); manga_id CHAR(36); tanpa status baca
reviews	id, user_id, manga_id, rating, comment, created_at	Ulasan dan rating pengguna
reading_progress	user_id, manga_id, manga_title, image_url, read_status, data chapter terakhir (id, nomor, judul, bahasa), last_read_at, total_chapters, completed_chapters, progress_percent, updated_at	Satu baris per pengguna dan manga (primary key user_id dan manga_id); menyimpan status baca dan progres
reading_history	id, user_id, manga_id, chapter_id, chapter_number, chapter_title, current_page, total_pages, first_read_at, last_read_at	Unik per (user_id, manga_id, chapter_id); dasar ranking Trending

Data disimpan pada volume Docker mysql_data. Cache katalog tidak disimpan di MySQL, melainkan di memori backend, sehingga hilang saat container backend dimulai ulang.

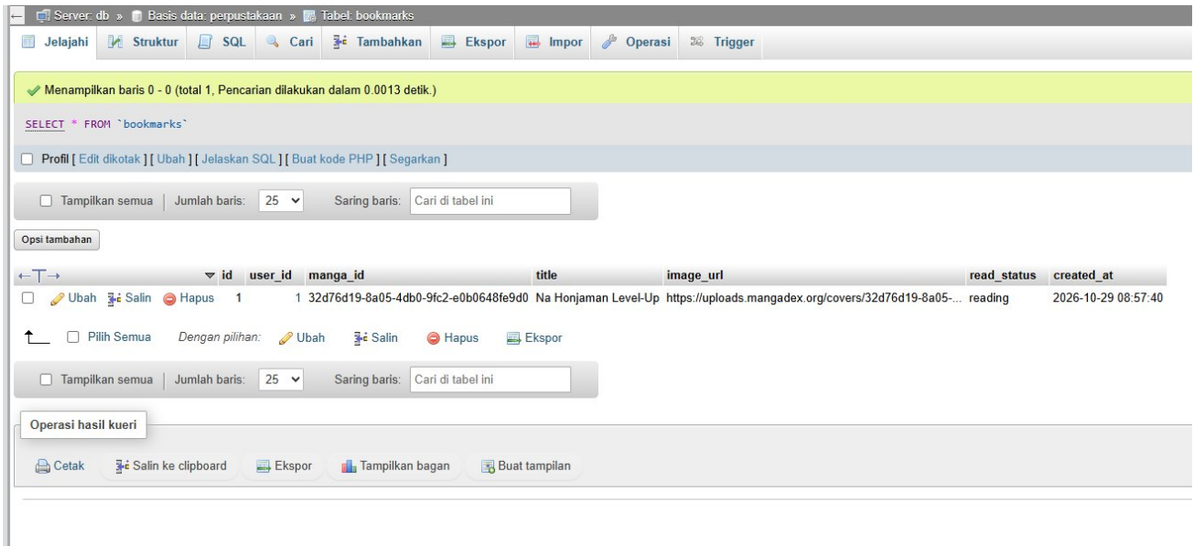
Tabel favorit didefinisikan di perpustakaan.sql. Pada database yang sudah pernah berjalan, tabel ini tidak terbentuk otomatis karena skrip inisialisasi hanya berjalan pada volume yang masih kosong dan migrate_mangadex.sql tidak memuatnya. Untuk database lama, jalankan definisi tabel berikut satu kali (misalnya lewat phpMyAdmin), atau buat ulang volume bila datanya boleh dihapus:

```
CREATE TABLE IF NOT EXISTS favorit (
  id int(11) NOT NULL AUTO_INCREMENT,
```

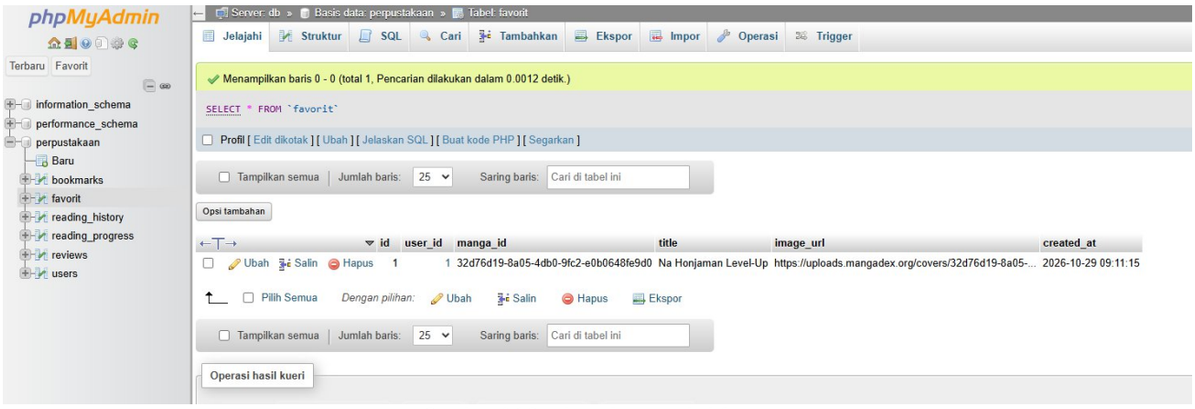
```
user_id int(11) NOT NULL,
manga_id char(36) NOT NULL,
title varchar(255) NOT NULL,
image_url varchar(500) DEFAULT NULL,
created_at timestamp NOT NULL DEFAULT current_timestamp(),
PRIMARY KEY (id),
UNIQUE KEY uniq_favorit_user_manga (user_id, manga_id),
CONSTRAINT fk_favorit_user FOREIGN KEY (user_id) REFERENCES users (id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```



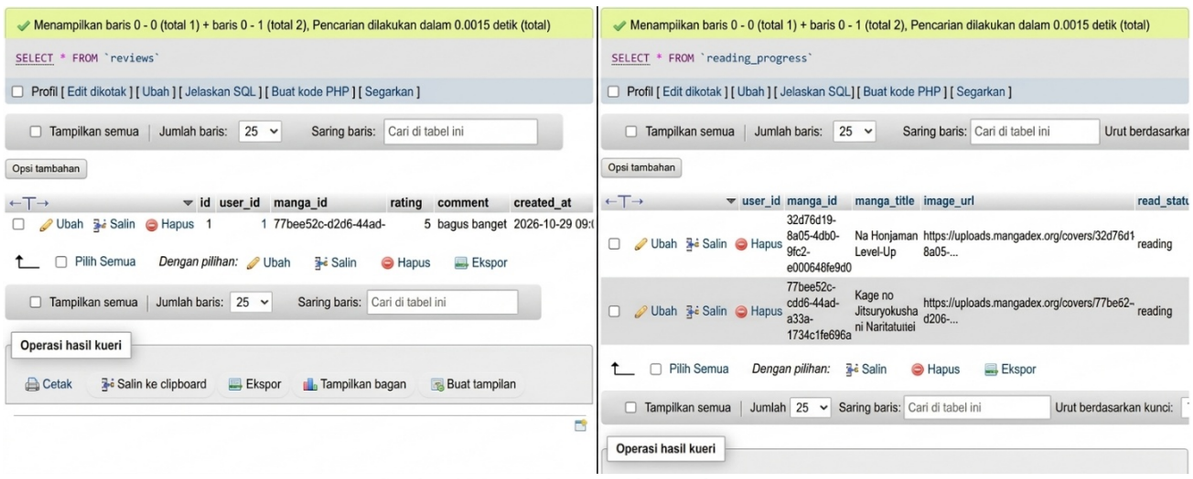
Gambar 34 Daftar enam tabel di phpMyAdmin



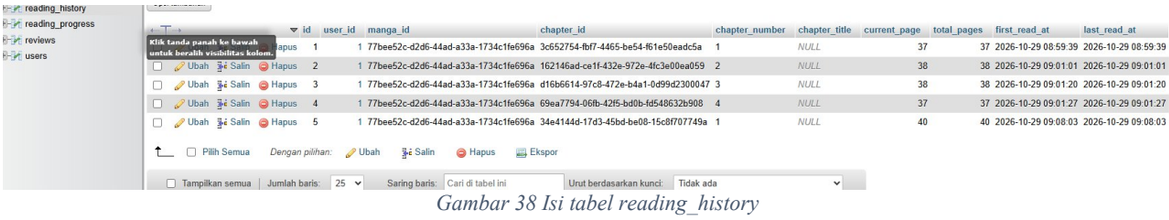
Gambar 35 Struktur tabel bookmarks



Gambar 36 Struktur dan isi tabel favorit



Gambar 37 Isi tabel reviews dan reading_progress



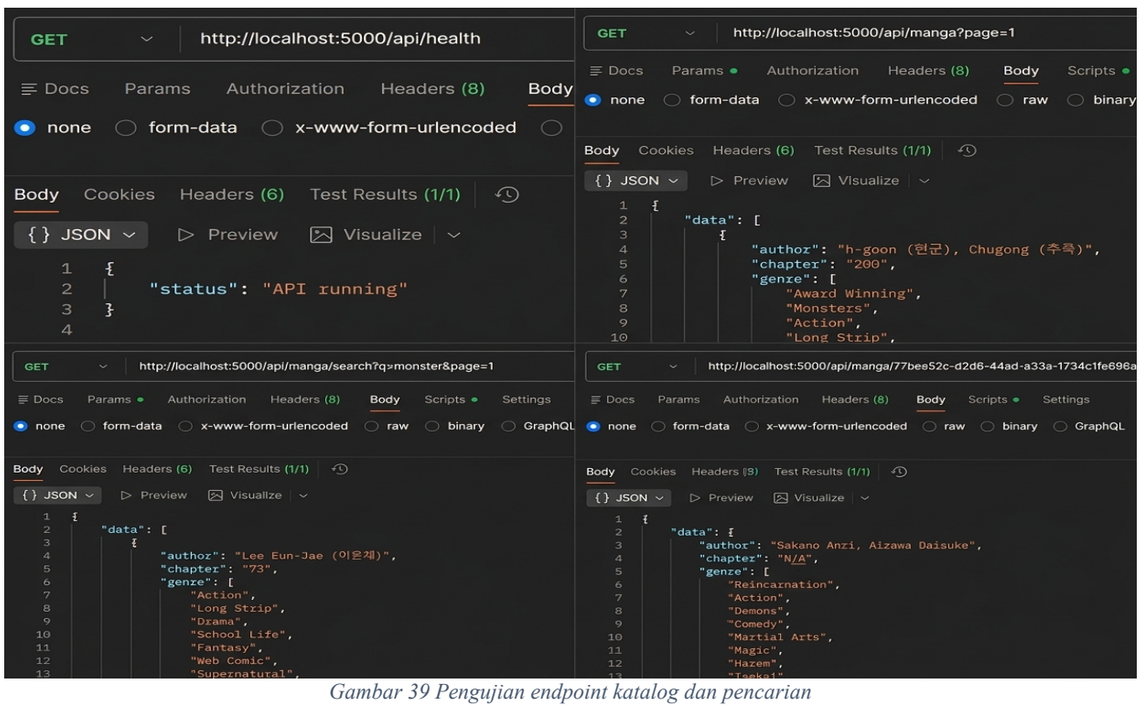
Gambar 38 Isi tabel reading_history

7. Endpoint API setelah update

Base URL lokal adalah http://localhost:5000. Endpoint dikelompokkan sebagai berikut, beserta capture hasil pengujiannya (kelompok a sampai e). Kelompok f sampai h hanya berisi daftar endpoint dan fungsi singkat.

a. Katalog dan pencarian

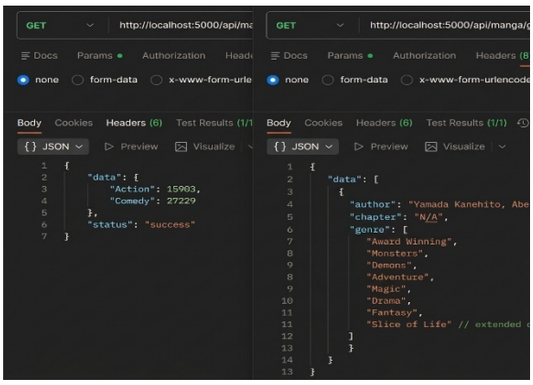
Endpoint	Fungsi	Catatan
GET /api/health	Pemeriksaan API	-
GET /api/manga?page=	Daftar manga katalog	Cache katalog maksimal 24 jam
GET /api/manga/search?q=&page=	Pencarian manga berdasarkan judul	Diteruskan ke MangaDex
GET /api/manga/{manga_id}	Detail manga	manga_id berupa UUID MangaDex



Gambar 39 Pengujian endpoint katalog dan pencarian

b. Genre

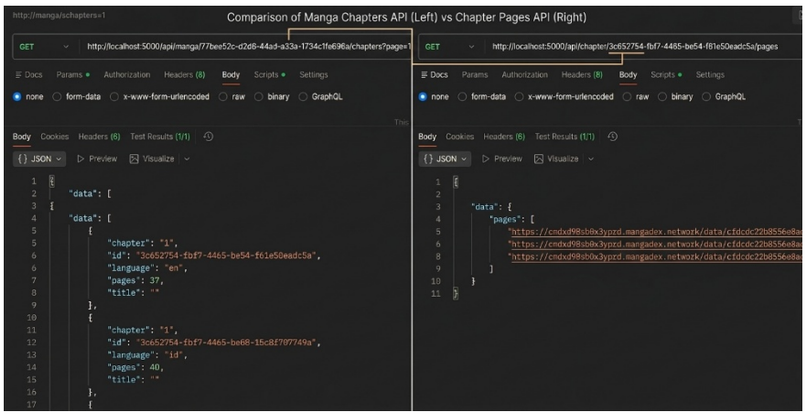
Endpoint	Fungsi	Catatan
GET /api/manga/genre-counts?genres=	Total manga per genre	Genre dengan total nol tidak ditampilkan
GET /api/manga/genre/{genre_name}?page=	Manga berdasarkan genre, 25 per halaman	Respons menyertakan total



Gambar 40 Pengujian endpoint genre

c. Chapter dan reader

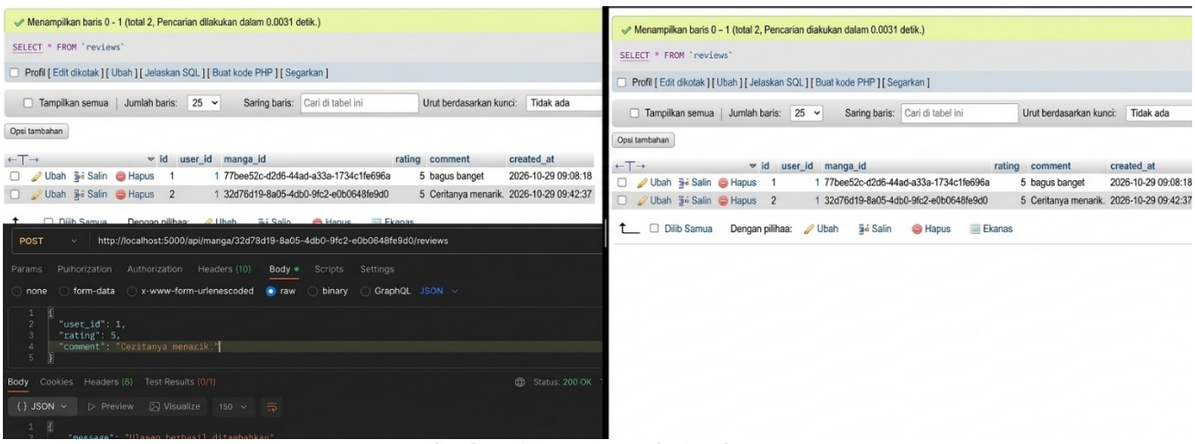
Endpoint	Fungsi	Catatan
GET /api/manga/{manga_id}/chapters	Daftar chapter bahasa Indonesia dan Inggris	Parameter page dan language (id/en); maksimal 100 per halaman
GET /api/chapter/{chapter_id}/pages	URL gambar halaman chapter	Gambar dimuat dari MangaDex, tidak disimpan



Gambar 41 Pengujian endpoint chapter dan reader

d. Ulasan

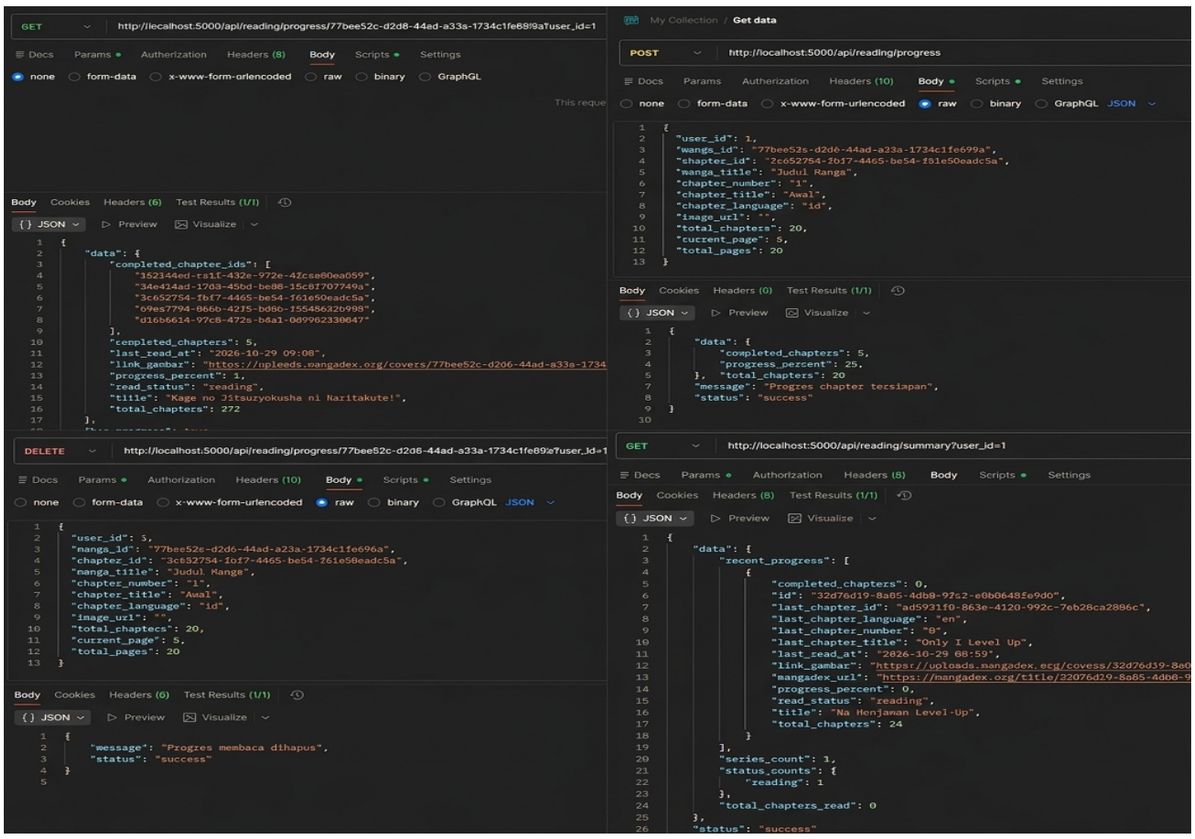
Endpoint	Fungsi	Catatan
GET /api/manga/{manga_id}/reviews	Melihat ulasan manga	Tersimpan di MySQL
POST /api/manga/{manga_id}/reviews	Mengirim rating dan komentar	Tersimpan di MySQL



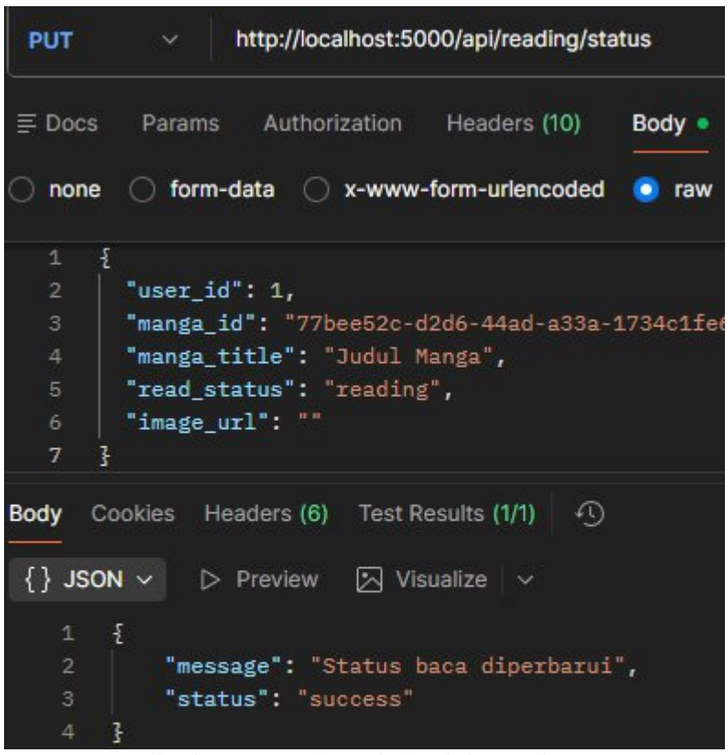
Gambar 42 Pengujian endpoint ulasan

e. Progres baca dan Trending

Endpoint	Fungsi	Catatan
GET /api/reading/progress/{manga_id}	Progres, chapter terakhir, dan chapter selesai	-
POST /api/reading/progress	Menyimpan chapter dan posisi halaman baca	-
DELETE /api/reading/progress/{manga_id}	Menghapus progres dan riwayat baca manga	-
GET /api/reading/summary	Ringkasan progres baca pengguna	-
PUT /api/reading/status	Mengubah status baca manga (plan, reading, completed)	Memperbarui reading_progress dan bookmarks; 401 bila user_id tidak dikirim
GET /api/manga/trending?period=	Ranking Trending (daily, weekly, monthly)	Dihitung dari reading_history dalam 1, 7, atau 30 hari



Gambar 43 Pengujian endpoint progres baca dan Trending



Gambar 44 Pengujian endpoint ubah status baca

Mulai kelompok f sampai akhir, sisa endpoint kami paparkan dalam bentuk tabel yang hanya berisi endpoint dan fungsi singkatnya, tanpa gambar seperti kelompok a sampai e sebelumnya. Kelompok yang dilengkapi capture adalah yang berkaitan langsung dengan fitur baru pada update ini, sedangkan akun, favorit, dan bookmark hanya didaftarkan agar dokumentasi API tetap lengkap.

f. Akun

Endpoint	Fungsi
POST /api/register	Registrasi akun baru
POST /api/login	Login dengan email dan password
DELETE /api/delete-account	Menghapus akun (verifikasi email dan password)

g. Favorit

Endpoint	Fungsi
POST /api/favorites	Menambah manga ke favorit
GET /api/favorites?user_id=	Daftar favorit pengguna
GET /api/favorites/{manga_id}?user_id=	Mengecek apakah manga sudah difavoritkan
DELETE /api/favorites/{manga_id}?user_id=	Menghapus manga dari favorit

h. Bookmark

Endpoint	Fungsi
POST /api/bookmark	Menyimpan manga ke rak buku
GET /api/bookmark?user_id=	Melihat semua bookmark pengguna
GET /api/bookmark/{manga_id}?user_id=	Mengecek apakah manga sudah dibookmark
PUT /api/bookmark/{manga_id}	Mengubah status baca (plan, reading, completed)
DELETE /api/bookmark/{manga_id}?user_id=	Menghapus bookmark

Implementasi CI/CD

1. Tujuan dan alur

Sebelumnya image mangatsu-api dan mangatsu-web dibangun dan diunggah ke Docker Hub secara manual dari komputer. Dengan CI/CD, proses itu dipindahkan ke GitHub Actions: setiap ada push ke repository (branch apa pun) atau saat workflow dijalankan manual lewat tombol Run workflow, GitHub otomatis mengambil kode, membangun Flutter Web, membangun kedua image, lalu mengirimnya ke Docker Hub. File workflow berada di repository carlos1739/mangatsu dengan nama ci-cd.yml. Perlu diingat, bagian CD pada workflow ini hanya sampai mengirim image ke Docker Hub dan belum berupa deployment otomatis. Cakupan dan batasannya dijelaskan pada bagian 6.

2. Flowchart

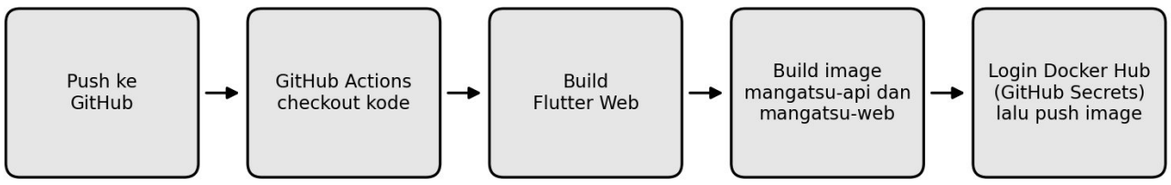


Diagram 4 Alur CI/CD Mangatsu

3. GitHub Secrets

Agar workflow dapat masuk ke Docker Hub tanpa mencantumkan kredensial di file konfigurasi, repository menyimpan dua GitHub Secrets: DOCKERHUB_USERNAME untuk username Docker Hub dan DOCKERHUB_TOKEN untuk access token. Gunakan access token, bukan password akun Docker Hub. Untuk menambahkannya, buka Settings → Secrets and variables → Actions → New repository secret, lalu buat kedua secret tersebut dengan nama yang sama.

4. Isi file workflow

Berikut isi file ci-cd.yml yang kami pakai.

```
name: Build and Push Docker Images

on:
  push:
    branches:
      - "**"
  workflow_dispatch:

permissions:
  contents: read

jobs:
  build-and-push:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Set up Flutter
        uses: subosito/flutter-action@v2
        with:
          channel: stable
          cache: true
```

```
- name: Build Flutter web
  working-directory: project_manga
  run: |
   flutter pub get
   flutter build web --release

- name: Set up Docker Buildx
  uses: docker/setup-buildx-action@v3

- name: Login to Docker Hub
  uses: docker/login-action@v3
  with:
    username: ${{ secrets.DOCKERHUB_USERNAME }}
    password: ${{ secrets.DOCKERHUB_TOKEN }}

- name: Build and push API image
  uses: docker/build-push-action@v6
  with:
    context: ./backend_manga
    push: true
    cache-from: type=gha,scope=api
    cache-to: type=gha,scope=api,mode=max
    tags: |
      carlossiregar/mangatsu-api:sha-${{ github.sha }}
      ${{ github.ref == 'refs/heads/main' && 'carlossiregar/mangatsu-api:latest' || '' }}
      ${{ github.ref == 'refs/heads/main' && 'carlossiregar/mangatsu-api:1.1' || '' }}

- name: Build and push web image
  uses: docker/build-push-action@v6
  with:
    context: ./project_manga
    push: true
    cache-from: type=gha,scope=web
    cache-to: type=gha,scope=web,mode=max
    tags: |
      carlossiregar/mangatsu-web:sha-${{ github.sha }}
      ${{ github.ref == 'refs/heads/main' && 'carlossiregar/mangatsu-web:latest' || '' }}
      ${{ github.ref == 'refs/heads/main' && 'carlossiregar/mangatsu-web:1.1' || '' }}
```

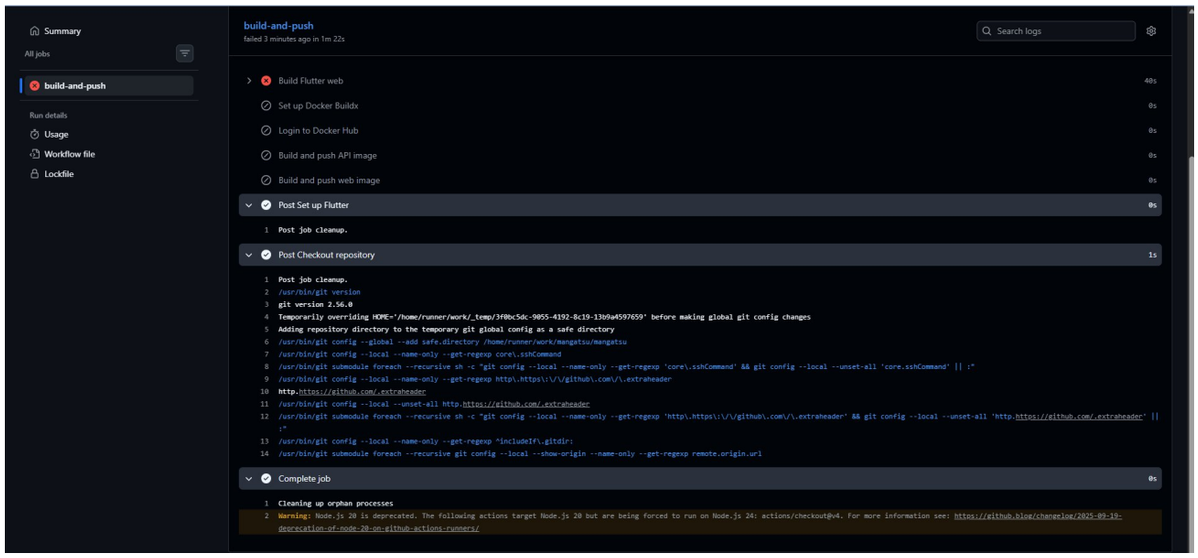
Penjelasan langkah-langkah pada job build-and-push:

Langkah	**Fungsi**
Checkout repository	Mengambil kode project dari GitHub ke runner.
Set up Flutter	Memasang Flutter (channel stable) di runner.
Build Flutter web	Menjalankan `flutter pub get` dan `flutter build web --release` di folder project_manga.
Set up Docker Buildx	Menyiapkan Docker Buildx untuk membangun image.
Login to Docker Hub	Login ke Docker Hub memakai DOCKERHUB_USERNAME dan DOCKERHUB_TOKEN dari GitHub Secrets.
Build and push API image	Membuild image dari folder backend_manga lalu push sebagai carlossiregar/mangatsu-api.
Build and push web image	Membuild image dari folder project_manga lalu push sebagai carlossiregar/mangatsu-web.

Setiap image diberi tag sha-<kode commit>. Tag latest dan 1.1 hanya ditambahkan bila build berasal dari branch utama, sesuai kondisi github.ref pada file di atas.

5. Bukti eksekusi

Pada percobaan pertama, workflow kami gagal. Job build-and-push berhenti setelah 1 menit 22 detik di langkah Build Flutter web (40 detik) yang bertanda silang merah. Karena langkah itu gagal, empat langkah sesudahnya (Set up Docker Buildx, Login to Docker Hub, Build and push API image, dan Build and push web image) tidak dijalankan dan ditandai dilewati, sehingga belum ada image yang terkirim ke Docker Hub. Peringatan kuning di bagian bawah log menyebut Node.js 20 pada actions/checkout@v4 sudah deprecated; itu hanya peringatan dan bukan penyebab gagalnya job. Kegagalan ini kami perbaiki lewat commit "Fix font asset filename for CI build", yaitu penyesuaian nama file font pada asset, lalu workflow dijalankan lagi.



Gambar 45 Workflow gagal di langkah Build Flutter web

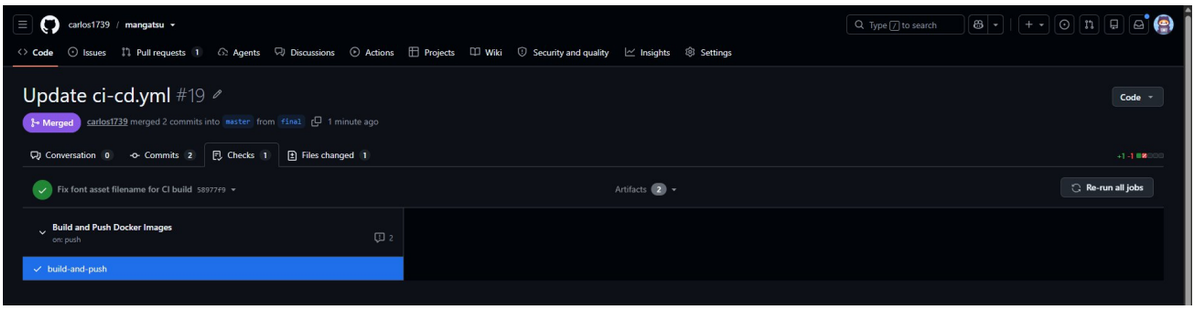
Setelah masalah pada proses build Flutter Web diperbaiki di atas, kami menjalankan workflow kembali. Run kedua selesai dalam 1 menit 55 detik dengan status berhasil. Semua tahap berjalan, termasuk build Flutter Web serta build dan push image API dan web ke Docker Hub. Hasil ini membuktikan proses build dan publish image berhasil. Buktinya bisa dilihat pada gambar di bawah ini.



Gambar 46 Workflow berhasil, semua langkah selesai

Hasil di GitHub.

Gambar berikut menunjukkan pull request #19 “Update ci-cd.yml” di repository carlos1739/mangatsu yang berstatus Merged: dua commit dari branch final digabung ke master. Pada tab Checks, check “Fix font asset filename for CI build” (commit 58977f9) berstatus hijau, dan job Build and Push Docker Images dijalankan karena push (on: push). Tercatat 2 anotasi peringatan pada run ini. Warning tersebut hanya muncul pada langkah build web, sedangkan langkah lainnya tetap berhasil berjalan.



Gambar 47 Pull request #19 sudah di-merge dan check berhasil

6. Cakupan dan batasan CI/CD

Workflow `ci-cd.yml` pada laporan ini baru mengotomatiskan proses build dan publish image. Supaya klaim CI/CD tidak disalahartikan, cakupannya kami rinci pada tabel berikut, mulai dari yang sudah berjalan sampai yang belum ada.

Aspek	Status	Keterangan
Build Flutter Web	Sudah ada	Langkah Build Flutter web menjalankan `flutter pub get` dan `flutter build web --release` pada setiap run.
Build image API dan web	Sudah ada	Dua image dibangun dari backend_manga/Dockerfile dan project_manga/Dockerfile.
Push ke Docker Hub	Sudah ada	Setiap build diberi tag sha-<kode commit>. Tag latest dan 1.1 hanya diberikan untuk branch main.
`flutter analyze`	Belum ada	Workflow tidak punya langkah analisis statis.
`flutter test`	Belum ada	Workflow tidak punya langkah test, dan file test Flutter tidak ditemukan di project_manga.
Test backend	Belum ada	Workflow tidak punya langkah test, dan file test backend tidak ditemukan di backend_manga.
Trigger pull request	Belum ada	Workflow hanya jalan saat push ke semua branch (`on: push`) atau saat dijalankan manual (`workflow_dispatch`). Tidak ada trigger `pull_request`.
Deployment otomatis	Belum ada	Tidak ada langkah yang menjalankan aplikasi di server. Hasil pipeline berhenti sebagai image di Docker Hub.
docker-compose.yml	Build lokal	Compose masih dapat membangun image dari source di komputer sendiri, dan tidak bergantung pada image di Docker Hub.

Dengan cakupan itu, run yang berstatus hijau hanya berarti Flutter Web berhasil dibangun dan kedua image berhasil dikirim ke Docker Hub. Run hijau belum menunjukkan bahwa kode lolos analisis

atau pengujian otomatis, dan belum berarti aplikasi sudah terpasang di server. Karena itu istilah CI/CD di laporan ini kami pakai dalam arti terbatas: CI sebatas build otomatis, sedangkan CD berhenti sampai publish image.

Pengujian fungsi tetap kami lakukan manual lewat aplikasi, phpMyAdmin, dan pengujian endpoint pada bagian Update Perubahan. Untuk menjalankan aplikasi, perintahnya tetap `docker compose up --build -d` dari folder project. Hal yang masih bisa ditambahkan ke depan adalah `flutter analyze`, `flutter test`, test backend, trigger pull request, dan deployment otomatis.

7. Keterkaitan Before/After dengan bukti pipeline

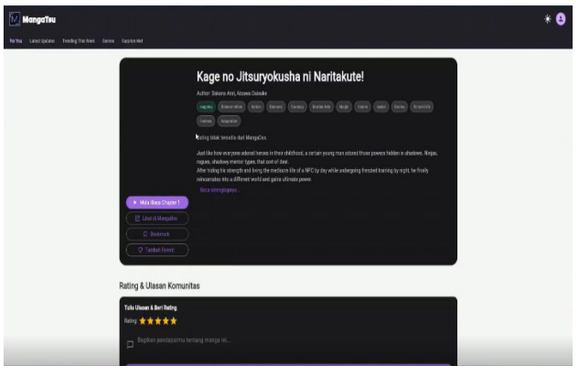
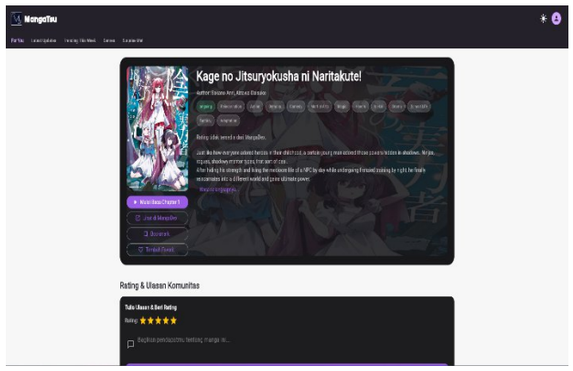
Bagian ini menunjukkan perubahan URL gambar cover pada halaman detail manga. Run GitHub Actions dan tag image Docker Hub dicocokkan berdasarkan commit SHA masing-masing.

Bukti	Before (versi awal)	After (setelah update)
Commit SHA (7 digit)	4d166d4	665ddc0
Commit SHA lengkap	4d166d494d0bc269a866ab5de86a6f0ba4505421	665ddc039d3fa918b0bea2a43a7ea8a7b9ce2ef4
Run GitHub Actions	Run 42996450337	Run 43124106201
Status run	Berhasil	Berhasil
Tag image mangatsu-api	carlossiregar/mangatsu-api:sha-4d166d494d0bc269a866ab5de86a6f0ba4505421	carlossiregar/mangatsu-api:sha-665ddc039d3fa918b0bea2a43a7ea8a7b9ce2ef4
Tag image mangatsu-web	carlossiregar/mangatsu-web:sha-4d166d494d0bc269a866ab5de86a6f0ba4505421	carlossiregar/mangatsu-web:sha-665ddc039d3fa918b0bea2a43a7ea8a7b9ce2ef4

Untuk memastikan versi Before dan After bisa ditelusuri, kami mencocokkan commit, run GitHub Actions, dan tag image di Docker Hub. Versi Before menggunakan commit 4d166d494d0bc269a866ab5de86a6f0ba4505421, dengan run GitHub Actions 42996450337 yang berstatus berhasil. Versi After menggunakan commit 665ddc039d3fa918b0bea2a43a7ea8a7b9ce2ef4, dengan run terbaru 43124106201 yang juga berhasil.

Pada kedua run, GitHub Actions membangun Flutter Web, lalu membangun dan mengirim image API serta web ke Docker Hub. Setiap image diberi tag yang memuat SHA commit, sehingga versinya dapat dicocokkan dengan run terkait. Untuk memeriksa hasil After, kami menarik image dengan tag SHA tersebut dan menjalankannya menggunakan Docker Compose. Dari pemeriksaan status container, image API dan web berhasil berjalan.

Kami kemudian membandingkan halaman detail manga yang sama pada kedua versi dengan ukuran layar yang sama. Perbandingan ini memperlihatkan perubahan pada tampilan cover. Bukti run yang berhasil menunjukkan proses build dan pengiriman image selesai; pengecekan fungsi aplikasi tetap kami lakukan secara terpisah karena workflow ini belum menjalankan test otomatis atau deployment ke server.

Sebelum Pembaruan	Sesudah Pembaruan
	

Gambar 48 Perbandingan halaman detail manga, Before (kiri) dan After (kanan)

Rekaman proses pipeline tersedia pada Video 2 di bagian 5.